ZISK WHITEPAPER[1]

ZISK CORE TEAM

Abstract. We present ZisK, a STARK-based zero-knowledge virtual machine (zkVM) that executes programs compiled from standard instruction set architectures and produces succinct, transparent proofs of their correct execution. ZisK targets the RISC-V ISA, allowing developers to use existing Rust and C toolchains without modification. Moreover, ZisK's modular architecture enables the addition of optimized constraint systems for expensive operations such as elliptic-curve arithmetic and bulk memory access, making it practical to prove complex computations at scale.

This paper provides a precise specification of the ZisK design: its instruction set architecture and execution semantics, the arithmetization framework, the circuit architecture with the relation enforced by every chip and bus, the continuation mechanism, the recursive aggregation system, and the proving backend.

Contents

1. Introduction **1**
2. Instruction Set Architecture **4**
3. Arithmetization Framework **13**
4. ZisK Design **19**
5. Continuations **35**
6. Recursion **41**
7. The ZisK Statement **53**
8. Proving Backend **60**
9. Soundness: Definitions, Deviations, and Proof Obligations **65**
References **73**
Appendix **75**

1. Introduction

Recent advances in zero-knowledge (ZK) proving systems [GWC19], [Ben+18a], [DP23], [HLP24] have opened the door to a new class of cryptographic applications: systems that can verify the correct execution of an arbitrary computation without revealing its private inputs, in time and space sublinear in the original computation. These properties underpin a rapidly expanding class of applications in blockchain scalability—such as validity rollups, verifiable coprocessors, and trustless cross-chain bridges—as well as an increasingly broad range of off-chain computation protocols.

1.1. Background and Motivation.

Early ZK systems required developers to express computations directly as arithmetic circuits using frameworks such as Circom [Bel+23] or Halo2

[1]This document describes ZisK v0.16.1 [48cf7cc – 2026-03-20].
Source code available at `github.com/0xPolygonHermez/zisk`.

[Ele22]. This low-level approach offers the greatest flexibility —developers can hand-tune constraint counts and choose their proof backend— but comes at a significant cost: every new computation requires custom circuit design, security review, and ongoing maintenance. The productivity model is fundamentally different from standard software development, limiting who can build ZK-enabled applications.

Zero-knowledge virtual machines (zkVMs) address this by interposing a fixed, general-purpose circuit between the developer and the proving system. The circuit verifies the correct execution of programs targeting a standard instruction set architecture (ISA), so developers can write programs in familiar languages like Rust or C and obtain proofs automatically, without writing a single constraint. The challenge for zkVM designers is then to make this general-purpose circuit as efficient as possible.

Existing zkVMs navigate a fundamental trade-off between developer experience and proving efficiency. Systems built around *ZK-native ISAs* (e.g., Cairo [GPR21] or Valida [Lit24]) define instruction sets whose semantics align naturally with finite-field arithmetic, enabling compact constraint representations but requiring developers to adopt a new programming model. Systems targeting *standard ISAs* such as RISC-V [RIS19] (e.g., RISC Zero [RIS23], SP1 [Suc24] or Nexus [Nex24]) preserve familiar toolchains and allow any RISC-V binary to run, but must constrain a general-purpose register machine whose operations —integer division, bitwise logic, memory addressing— are expensive to encode as polynomial constraints.

1.2. **ZisK.**

This paper introduces ZisK, a zkVM designed to make zero-knowledge proofs of general-purpose computation practical at production scale. ZisK targets the RISC-V ISA, preserving compatibility with mature toolchains such as LLVM and Rust so that programs can be proved without changes to existing development workflows. This positions ZisK in the standard-ISA camp, but with a deliberate architecture that treats proving performance as a first-class concern: rather than accepting the overhead of a general-purpose CPU circuit, ZisK uses a modular design where expensive operations are handled by dedicated, independently optimized constraint systems.

A central motivation for ZisK is *extensibility*. Application domains such as blockchain execution environments rely on a specific, known set of expensive primitives —elliptic-curve operations, hash functions, bulk memory transfers. A zkVM designed for these domains should be able to support those primitives natively, with proofs that are tight to the operation's intrinsic complexity. ZisK is structured so that such optimized components can be added as self-contained modules without modifying the rest of the system, making it practical to tailor the prover to a specific application's

workload. Building on STARKs [Ben+18b] further ensures that this is achieved without a trusted setup and with post-quantum security.

1.3. **Paper Overview.**

Section 2 defines the ZisK ISA: execution environment, machine state, instruction format, the full instruction set, and the formal step semantics. Section 3 presents the arithmetization framework: AIRs, bus interactions, and LogUp-based circuit satisfiability. Section 4 describes the chip architecture and bus system, including the memory-consistency relation, the encoding of 64-bit words over Goldilocks, and the schema and semantic relation of every bus. Section 5 explains trace splitting, the formal notion of a segment, and how continuation buses chain segment boundaries. Section 6 covers the recursive aggregation tree, the challenge derivation, and root-level bus balancing. Section 7 states what ZisK proves as an execution relation with a knowledge-soundness target. Section 8 fixes the proving backend and its parameters. Section 9 gives the topology of all proof-system calls, the deviations from a vanilla construction, the soundness sketch, and the ledger of remaining proof obligations.

1.4. **Notation.**

For the remainder of the paper, we fix $\mathbb{F}$ to be a finite field of prime order p. In particular, we use the Goldilocks prime field $p = 2^{64} - 2^{32} + 1$, though this choice is not fundamental to the overall design and any appropriate prime would suffice. When needed, we identify field elements with integers in the range $[0, p)$.

We also fix a finite extension field $\mathbb{K}$ of $\mathbb{F}$, primarily for the security of the underlying proving system; concretely, ZisK targets 128-bit security. In particular, we use the third Goldilocks extension $\mathbb{F}[X]/(X^3 - X - 1)$, whose order $|\mathbb{K}| = p^3 \approx 2^{192}$ gives enough room to drive the proving system's soundness error well below 2^{-128}, while still allowing efficient arithmetic. Again, this choice is not fundamental to the design.

We denote by $\mathbb{F}[X]$ the ring of univariate polynomials over $\mathbb{F}$ and by $\mathbb{F}[X_1, ..., X_n]$ the ring of n-variate polynomials over $\mathbb{F}$. When needed, we write $\mathbb{F}^{<N}[X]$ to refer to the subset of polynomials of degree strictly less than N. We use $\mathbb{F}^{n\times m}$ to denote the set of n-by-m matrices over $\mathbb{F}$. The same notation applies to $\mathbb{K}$.

When the additive structure is the only thing that matters, we sometimes write $\mathbb{Z}_p$ for the abelian group of integers modulo p —the same underlying set as $\mathbb{F}$, viewed as a $\mathbb{Z}$-module rather than a field. Vectors over this group are denoted $\mathbb{Z}_p^n$.

We use the notation $[a, b]$ to denote the set of integers from a to b, inclusive; and $[a, b)$ to denote the set of integers from a to b, inclusive of a and exclusive of b. When $a = 1$ and $b > 1$, we write $[b]$ instead of $[1, b]$.

2. Instruction Set Architecture

The ZisK instruction set architecture (ZisK ISA) specifies the computational model that ZisK proves correct: the execution model, the memory layout, the instruction format, and the full set of operations. Because the ZisK ISA is a *virtual* ISA — not targeting any physical hardware — its design can prioritize properties that simplify zero-knowledge proof generation rather than properties that suit a particular microarchitecture.

The ZisK ISA is designed around two goals. The first is compatibility: ZisK targets the RISC-V ISA so that programs written in standard languages and compiled with existing toolchains can be run without modification. The second goal is efficiency: operations that would be expensive to constrain step-by-step in the main execution circuit —elliptic-curve arithmetic, hash functions, bulk memory transfers— are exposed as *precompile instructions*, each handled by a dedicated, optimized sub-circuit. This combination gives the developer accessibility of a standard ISA with the proving performance of a custom one.

Remark (RISC-V to ZisK transpilation). *Although programs are written and compiled as standard RISC-V binaries, ZisK does not constrain RISC-V execution directly. Before proving begins, the binary is* transpiled *off-chain into an equivalent program in the ZisK ISA, and it is the transpiled program that is committed to and executed by the VM. The transpilation is a deterministic, one-to-one rewriting from RISC-V instructions into one or more ZisK instructions; it is performed by a trusted off-chain tool and is not itself part of the proof. We therefore treat the transpiler as part of the trusted computing base.*

The rest of this section describes the execution environment (Section 2.1), the machine state (Section 2.2), and the formal execution model (Section 2.3).

2.1. Execution Environment.

Programs in ZisK are executed inside a *virtual machine* (VM). The VM is a *state machine*: execution proceeds as a sequence of discrete steps, each step transforming the current *guest state* according to one instruction. The VM is hosted on a physical machine, called the *host*, that maintains an external *host state* separate from the guest. The ZisK ISA provides specific interfaces through which the guest program can read from and write to the host state.

This host/guest separation is central to ZisK's trust model. The guest program runs in a fully deterministic, sandboxed environment: every value it operates on is either derived from the initial program ROM (Section 2.2.1), loaded from memory, or explicitly received from the host through a typed channel. The host can supply:

- **Public Inputs**. Values known to both prover and verifier, written into the input data region at the start of execution and read-only thereafter.
- **Private Inputs (Hints)**. Non-deterministic values provided by the host at runtime. The guest may request a hint and use its value, but must verify any properties it relies on —ZisK treats hints as unconstrained. A hint that is never verified proves nothing.

The key property of this model is that *everything the guest touches is constrained by the VM's arithmetization*. Memory accesses are constrained by the memory argument; program code is validated by the ROM bus on every fetch; and public outputs become explicit proof data. The host state, by contrast, lies outside and is trusted by convention.

2.2. **State.**

As defined before, the VM execution state is everything that changes from one step to the next. It is divided into the *guest state* —manipulated by the program— and the *host state* —maintained externally and supplied before or during execution. The guest state comprises the program ROM (read-only), the program counter `pc`, data memory (read/write), and the public outputs. The host state comprises the input stream and the hint stream.

2.2.1. *Program ROM.*

ZisK can execute any program compiled to its ISA. A program consists of two parts:

- **The Program Code**. The program code maps 32-bit instruction addresses to ZisK instructions. It is loaded into the VM before execution begins.
- **The ROM Data**. The ROM data is an immutable data segment holding program constants (e.g., string literals, lookup tables, and static data embedded in the binary). It occupies a dedicated region of the address space and can be read but never written by the program.

Both parts are determined entirely by the program binary: they are identical across every execution of that program, regardless of the inputs provided. Once the program is fixed, neither the code nor the ROM data can change — no instruction can modify them and they play no role in the per-step state transitions of the machine. This separation of code from mutable data is the classical *Harvard architecture*: code and data live in distinct address spaces and are governed by different access rules.

From the proving system's perspective, the two parts are committed to and validated by distinct mechanisms. The program code is organised as a fixed table of instructions indexed by their addresses. Before any execution

begins, this table is hashed into a *program commitment* that is included in the verifier's context. At every step, the main circuit performs a table lookup against this committed table —retrieving the instruction at the current program counter and verifying it belongs to the ROM. No instruction can be substituted or forged without invalidating the commitment.

The ROM data is validated through a different mechanism. Rather than a separate lookup, it is treated as a region of *immutable memory* within a dedicated address space (Section 2.2.6). At initialization, the ROM data region is written once to establish its contents; thereafter only reads are permitted. From the running program's point of view, reads from ROM data are indistinguishable from reads from writable RAM — the immutability is a property enforced by the arithmetization, not by the ISA.

2.2.2. *Instruction Format.*

A ZisK program P is a finite partial map from 32-bit *instruction addresses* to *instructions*. An instruction is a fixed-format record; the program ROM stores exactly one such record per address, and the proof system commits to the whole table (Section 2.2.1). Every instruction performs one operation over two 64-bit input operands, producing a 64-bit output and a boolean flag:

$$(c, \text{flag}) = \text{op}(a, b), \quad a, b, c \in [0, 2^{64}), \text{flag} \in \{0, 1\}.$$

What varies between instructions is how a and b are obtained, where c is stored, and how the program counter advances.

The record has the following fields; boolean fields are packed into a single `flags` word in the committed ROM (Section 4.4.2).

`op` $\in [\mathbf{0}, \mathbf{2^8})$ the opcode (Section 2.2.3, Appendix B).

`a_src` $\in \{$**`C`**, **`MEM`**, **`IMM`**, **`REG`**$\}$ how operand a is obtained: the previous output c, an absolute memory address, an immediate, or a register.

`a_off`, **`a_imm1`** $\in [\mathbf{0}, \mathbf{2^{32}})$ for `REG` the register index; for `MEM` the absolute address; for `IMM` the low and high 32 bits of the immediate.

`b_src` $\in \{$**`C`**, **`MEM`**, **`IMM`**, **`REG`**, **`IND`**$\}$ how operand b is obtained; `IND` is an indirect load at address $a +$ `b_off`.

`b_off` $\in [-\mathbf{2^{31}}, \mathbf{2^{31}})$, **`b_imm1`** $\in [\mathbf{0}, \mathbf{2^{32}})$ register index, absolute address, low and high immediate bits, or the signed displacement of an indirect load.

`ind_w` $\in \{$**1**, **2**, **4**, **8**$\}$ access width in bytes for `IND` loads and `IND` stores.

`store` $\in \{$**`NONE`**, **`REG`**, **`MEM`**, **`IND`**$\}$ destination of the stored value.

`store_off` $\in [-\mathbf{2^{31}}, \mathbf{2^{31}})$ register index, absolute address, or signed displacement added to a for an `IND` store.

`store_pc` $\in \{$**0**, **1**$\}$ store the return address `pc + j2` instead of c.

`set_pc` $\in \{$**0**, **1**$\}$ take the next program counter from c (indirect jump).

j1, j2 $\in [-2^{31}, 2^{31})$ jump offsets, `j1` for a taken branch or indirect jump and `j2` for fall-through. For precompile instructions `j1` is instead a static *extended argument*.

m32 $\in \{0, 1\}$ the operation acts on the low 32 bits of the operands (the `_w` opcodes).

end $\in \{0, 1\}$ execution halts after this instruction.

The derived flags `is_external_op` (the operation is delegated to a sub-chip via the Operation Bus) and `is_precompiled` (the operation reads and writes memory itself) are functions of `op`.

The full ZisK instruction set is listed in Appendix B. The reference implementation is `zisk_ops.rs` the table is a faithful transcription of that file at v0.16.1 and is normative for this document.

2.2.3. *Operations.*

ZisK v0.16.1 has 54 opcodes plus 6 internal DMA variants. They fall into the following families, each proved by one instruction-executor chip (Section 4.2); the exact semantics of every opcode —the formula for c and for flag, including wrap-around, signedness, shift masking and division-by-zero conventions— is listed in Appendix B, which is a faithful transcription of `zisk_ops.rs` at v0.16.1 and is normative for this document.

- **Internal** (proved by Main): `flag`, `copyb`, and the invalid-instruction sentinel `halt`.
- **Binary** (Binary and BinaryAdd chips): 64-bit and 32-bit comparisons, minimum/maximum, addition, subtraction and bitwise logic; the comparison opcodes set flag to the comparison result.
- **Binary extension** (BinaryExtension chip): logical and arithmetic shifts in 64 and 32 bits, and sign extension from 8, 16 and 32 bits.
- **Arithmetic** (Arith chip): 64-bit and 32-bit multiplication with high and low halves, and division and remainder; division by zero sets flag.
- **Public output** (Main, bound globally): `pubout` declares public output word a to be b (Section 7).
- **Hints** (Main): the three `fcall_*` opcodes drive the host-side hint stream (Section 2.2.8).
- **Precompiles** (one chip each): Keccak-f, SHA-256 compression, Poseidon2, BLAKE2b, 256-bit addition, 256- and 384-bit modular arithmetic, secp256k1, secp256r1, BN254 and BLS12-381 curve and $\mathbb{F}_{p^2}$ operations, and the DMA bulk-memory operations. These read their operands from and write their results to memory (Section 4.2.4).

2.2.4. *Phantom Instructions.*

Most opcodes carry a proof obligation for the operation they perform, discharged by the chip named for it in Appendix B. A few opcodes are *phantom*: they leave the guest state unchanged apart from the trivial effects $c = 0$ or $c = b$, but may trigger side effects in the host state, such as advancing the hint stream. Phantom instructions appear in the main execution trace but carry no proof obligation beyond the Main chip's own constraints. They are `flag`, `copyb`, and the three `fcall_*` hint instructions; `halt` is the invalid-instruction sentinel and cannot appear in a valid trace.

Precompile dispatch follows a calling convention: the caller writes the parameters to memory, then issues the precompile opcode with b pointing to the parameter block. All parameter passing is done with ordinary instructions; only the precompile instruction itself carries a proof obligation, which is discharged by the corresponding precompile chip together with the memory argument.

2.2.5. *Program Counter.*

The program counter `pc` is a 32-bit special-purpose register storing the address of the instruction currently being executed. Its update rule is part of the step semantics (Section 2.3): it advances by `j2` unless the instruction sets the flag (branch taken: advance by `j1`) or has `set_pc` set (indirect jump: c + `j1`).

2.2.6. *Data Memory.*

Data memory is a *random access memory (RAM)* which supports read and write operations. It is separate from the program ROM and is used for mutable state during execution. The ZisK ISA defines a 32-bit byte address space for data memory, but the native unit of access is a 64-bit word (8 bytes). Memory cells are aligned to 8-byte boundaries, so valid addresses are multiples of 8. ZisK supports sub-word accesses (1, 2, or 4 bytes) by decomposing them into aligned 8-byte operations (Section 4.2.2).

To support execution of programs with different data assumptions and properties within the same architecture, we separate data memory into distinct regions, each with its own access semantics. Table 1 summarises the address space layout and the semantics of each region.

Region	Address Range	Semantics
Input Data	`[0x4000_0000, 0x8000_0000)`	[1GB] Host-supplied input; read-only. The first word, `0x4000_0000`, is the *free-input port* through which hints are delivered (Section 2.2.8).
ROM Data	`[0x8000_0000, 0x8800_0000)`	[128MB] Program constants, read-only
Writable RAM	`[0xA000_0000, 0xA001_0000)`	[64KB] System region: registers, UART and CSR registers
	`[0xA001_0000, 0xA002_0000)`	[64KB] Output data: public outputs written during execution; the following 64KB are reserved
	`[0xA003_0000, 0xC000_0000)`	[496MB] Program memory: general-purpose read/write RAM

TABLE 1. Memory layout of the ZisK ISA. The address space is divided into three main regions: input data, ROM data, and writable RAM.

The 32 general-purpose registers $x_0, ..., x_{31}$ occupy the start of the system region, each spanning 8 bytes, following the RISC-V ABI [RIS24]. Register x_0 is hardwired to zero. From the ISA's perspective, registers are addressed in the same numerical address space as data memory: a load or store

targeting the register region behaves exactly as if it targeted RAM, which keeps the ISA uniform and lets compiler-emitted code access registers through ordinary memory operations. Internally, however, ZisK handles the register region separately from the rest of memory for performance reasons; this distinction is invisible to the program but is described in Section 4.2.

2.2.7. *Public Outputs.*

The guest program can expose up to 32 values of 64 bits (2048 bits in total) as *public outputs*. During execution the guest writes them to the output-data region of RAM (Table 1) with ordinary stores; when the program returns to the boot code, the boot code reads the 32 words back from that region and issues one `pubout` instruction per word (Appendix B), with $a = i \in [0, 32)$ the index and b the word. The `pubout` instructions are what binds the proof's public statement to the execution: the public statement contains exactly these 32 words (Section 7), and nothing else about the input or the memory.

2.2.8. *Inputs and Hints.*

The host state maintains two streams through which the outside world communicates with the guest. The *input stream* is loaded into the input-data region before execution begins; from the proof's point of view its contents are *not* public: the public statement (Section 7) does not contain them, and a guest that wants to bind its outputs to its inputs must do so itself, e.g. by exposing a hash of the input as a public output. The *hint stream* provides non-deterministic values at runtime: the guest requests a hint with the `fcall` instructions and then reads the result through the *free-input port*, a load of width 8 at address `0x4000_0000` that returns the next hint value. The guest must verify any property of a hint it relies on, since ZisK treats hints as unconstrained. Formally, both streams are part of the host state H, and the only way their values enter the guest state is through memory reads of the input-data region, whose values the arithmetization leaves free (Section 4.2.2.1).

2.3. **Execution Model.**

The execution model defines how the VM transitions between states, what constitutes a valid execution, and which parts of an execution are exposed to the verifier. The W1 version of this document described the single-step map Step as a black box "fully determined by the ISA"; here we spell it out, since the soundness statement of Section 7 quantifies over it.

Memory as a function. We model data memory as a total function $\mathrm{mem} : [0, 2^{32}) \to [0, 2^8)$ from byte addresses to bytes. For a width $w \in \{1, 2, 4, 8\}$ and an address x we write $\mathrm{mem}[x]_w := \sum_{i=0}^{w-1} 2^{8i} \cdot \mathrm{mem}(x + i) \in [0, 2^{8w})$ for the little-endian w-byte read, and $\mathrm{mem}[x]_w \leftarrow v$ for the map that stores the w low-order bytes of v at $x, ..., x + w - 1$ and agrees with mem

elsewhere. Registers are memory: $x_i \coloneqq \mathrm{mem}[\mathrm{REG} + 8i]_8$ for $i \in [0, 32)$, where $\mathrm{REG} =$ `0xA000_0000` is the start of the system region (Table 1). Register x_0 is hardwired to zero at transpilation time (every read of x_0 is rewritten to the immediate 0 and every write to x_0 is dropped), so the program never accesses it.

Guest state. Since the program ROM is fixed before execution begins, the *guest state* at step t is

$$S_t = (\mathrm{pc}_t, c_t, \mathrm{mem}_t),$$

comprising the program counter, the previous instruction's output (which the next instruction may consume through the source `C`), and the data memory including the registers. The flag produced by an instruction is consumed only by the same instruction's control-flow rule, so it is not part of the state. The host state $H = (\mathrm{in}[], \mathrm{hint}[])$ is supplied externally: in[] is loaded into the input-data region before execution, and hint_t is the sequence of hint words delivered through the free-input port during step t.

Fetch. The instruction executed at step t is

$$\iota_t \coloneqq \mathrm{Fetch}_P(S_t) \coloneqq P[\mathrm{pc}_t],$$

the instruction stored in the program ROM at the current program counter. Execution *fails* if pc_t is not in the domain of P.

Operand resolution. Given a state $S = (\mathrm{pc}, c, \mathrm{mem})$ and an instruction ι, the operands are

$$a \coloneqq \begin{cases} c & \text{if } \iota.\mathrm{a_src} = \texttt{C} \\ \mathrm{mem}[\iota.\mathrm{a_off}]_8 & \text{if } \iota.\mathrm{a_src} = \texttt{MEM} \\ \iota.\mathrm{a_off} + 2^{32} \cdot \iota.\mathrm{a_imm1} & \text{if } \iota.\mathrm{a_src} = \texttt{IMM} \\ x_{\iota.\mathrm{a_off}} & \text{if } \iota.\mathrm{a_src} = \texttt{REG} \end{cases} \qquad b \coloneqq \begin{cases} c & \text{if } \iota.\mathrm{b_src} = \texttt{C} \\ \mathrm{mem}[\iota.\mathrm{b_off}]_8 & \text{if } \iota.\mathrm{b_src} = \texttt{MEM} \\ \iota.\mathrm{b_off} + 2^{32} \cdot \iota.\mathrm{b_imm1} & \text{if } \iota.\mathrm{b_src} = \texttt{IMM} \\ x_{\iota.\mathrm{b_off}} & \text{if } \iota.\mathrm{b_src} = \texttt{REG} \\ \mathrm{mem}[(a + \iota.\mathrm{b_off}) \bmod 2^{32}]_{\iota.\mathrm{ind_w}} & \text{if } \iota.\mathrm{b_src} = \texttt{IND} \end{cases}$$

where in the `IND` case the requirement $a < 2^{32}$ is part of the well-formedness of the step (the arithmetization enforces a's high word to be zero whenever an indirect address is formed). Reads of the free-input port —address `0x4000_0000` with width 8— return the next hint word hint_t instead of the memory contents; this is the only non-deterministic input to a step.

Operation. $(c', \mathrm{flag}) \coloneqq \iota.\mathrm{op}(a, b)$ as specified in Appendix B. For precompile opcodes the operation additionally has a *memory effect*: it reads its parameter block from mem at addresses derived from b and writes its results back, as described in Appendix B and in detail in Section 4.2.4. We write $\mathrm{mem}^{\mathrm{op}} \coloneqq \mathrm{Eff}_{\iota.\mathrm{op}}(\mathrm{mem}, a, b, \iota.\mathrm{j1})$ for the memory after the precompile's effect (with Eff the identity for all non-precompile opcodes); the fourth argument is the extended static argument that some DMA opcodes take from the `j1` field.

Store. The stored value is $v := ((\mathrm{pc} + \iota.\mathrm{j2}) \bmod 2^{32})$ if $\iota.\mathrm{store_pc} = 1$ and $v := c'$ otherwise, and

$$\mathrm{mem}' := \begin{cases} \mathrm{mem}^{\mathrm{op}} & \text{if } \iota.\mathrm{store} = \texttt{NONE} \\ \mathrm{mem}^{\mathrm{op}}[\mathrm{REG} + 8 \cdot \iota.\mathrm{store_off}]_8 \leftarrow v & \text{if } \iota.\mathrm{store} = \texttt{REG} \\ \mathrm{mem}^{\mathrm{op}}[\iota.\mathrm{store_off}]_8 \leftarrow v & \text{if } \iota.\mathrm{store} = \texttt{MEM} \\ \mathrm{mem}^{\mathrm{op}}[(a + \iota.\mathrm{store_off}) \bmod 2^{32}]_{\iota.\mathrm{ind_w}} \leftarrow v & \text{if } \iota.\mathrm{store} = \texttt{IND}. \end{cases}$$

Control flow. The next program counter is

$$\mathrm{pc}' := \begin{cases} (c' + \iota.\mathrm{j1}) \bmod 2^{32} & \text{if } \iota.\mathrm{set_pc} = 1 \\ (\mathrm{pc} + \iota.\mathrm{j1}) \bmod 2^{32} & \text{if } \iota.\mathrm{set_pc} = 0 \wedge \mathrm{flag} = 1 \\ (\mathrm{pc} + \iota.\mathrm{j2}) \bmod 2^{32} & \text{if } \iota.\mathrm{set_pc} = 0 \wedge \mathrm{flag} = 0 \end{cases}$$

and a valid instruction never has both $\iota.\mathrm{set_pc} = 1$ and $\mathrm{flag} = 1$ (the arithmetization enforces $\mathrm{flag} \cdot \mathrm{set_pc} = 0$). Note that `j1` doubles as the extended argument of precompile instructions; for those, $\mathrm{flag} = 0$ and $\mathrm{set_pc} = 0$, so `j1` never reaches the program counter.

Definition 1 (ZisK Step Predicate). Fix a program P. The *step predicate* $\mathrm{Step}_P(S, \iota, h, S')$ holds for states $S = (\mathrm{pc}, c, \mathrm{mem})$, $S' = (\mathrm{pc}', c', \mathrm{mem}')$, instruction ι, and hint sequence h if and only if $\iota = P[\mathrm{pc}]$ and, with $a, b, c', \mathrm{flag}, \mathrm{mem}', \mathrm{pc}'$ computed from (S, ι, h) by the operand-resolution, operation, store and control-flow rules above, the components of S' are exactly those values. We write $\mathrm{Step}_P(S, h) = S'$ for the (unique) S' satisfying the predicate, when it exists.

The predicate is total on well-formed inputs and deterministic given the hints: for every S with $\mathrm{pc} \in \mathrm{dom}(P)$ and every h there is exactly one S', and S' does not depend on h unless the instruction reads the free-input port. This is what makes ZisK's execution a function of $(P, \mathrm{in}, \mathrm{hint})$.

Initial state. Execution begins at the boot address $\mathrm{pc}_{\mathrm{BOOT}} = \texttt{0x1000}$ with memory initialised from the program binary and the input:

$$S_0 = (\mathrm{pc}_{\mathrm{BOOT}}, 0, \mathrm{mem}_0), \quad \mathrm{mem}_0 = \mathrm{ROMDATA}_P \cup \mathrm{in}[] \cup 0,$$

i.e. the ROM-data region holds the program constants, the input-data region holds the host-supplied input stream (word-aligned, preceded by the 8-byte free-input port), and every other address —including all registers— holds zero.

Terminal condition. Every ZisK program contains, at address $\mathrm{pc}_{\mathrm{END}} = \texttt{0x1004}$, the unique instruction with the `end` field set. A T-step execution is *successful* if $\mathrm{pc}_T = \mathrm{pc}_{\mathrm{END}}$ and Step_P holds at every step $t < T$; the boot code guarantees that $\mathrm{pc}_{\mathrm{END}}$ is reached with $c_T = 0$. Execution *fails* if a step violates the ISA's invariants —undefined program counter, out-of-range memory access, or the `halt` sentinel— or if the program counter never reaches $\mathrm{pc}_{\mathrm{END}}$. Only successful executions are provable.

Public outputs. The public outputs are not a separate component of the state. The guest writes them to the output-data region of RAM with ordinary stores, and the boot code, after the guest returns, reads the 32 words of that region and issues one `pubout` instruction per word (Appendix B). We write

$$\mathrm{out}_i(S_0, ..., S_T) := b_t \quad \text{where } t \text{ is the unique step with } \iota_t.\mathrm{op} = \texttt{pubout} \text{ and } a_t = i, \quad i \in [0, 32),$$

for the i-th public output of the execution; the boot code executes exactly one `pubout` per index. These 32 words are the only execution-dependent data in the proof's public statement (Section 7).

3. Arithmetization Framework

A proving system is naturally divided into two layers. The *frontend* takes a program and produces a structured mathematical representation — typically a system of polynomial constraints — that captures exactly what it means for the computation to be correct. The *backend* then takes that representation and produces a cryptographic proof of its satisfiability. Concretely, the backend performs a one-time *setup* that yields a pair $(\mathrm{pk}, \mathrm{vk})$: a *proving key* used to generate proofs of the frontend's constraint system, and a *verification key* that summarises the constraint system in a form the verifier can check against. Verification of a proof π against a public statement x is then the deterministic procedure $V(\mathrm{vk}, x, \pi) \in \{\mathrm{accept}, \mathrm{reject}\}$. The two layers are decoupled by design: a single frontend can target different backends, and improvements to either do not require redesigning the other.

ZisK's *arithmetization framework* is its frontend. The choice of arithmetization is consequential: it determines how circuits are structured, what operations are native, and how efficiently the backend can prove them. ZisK's arithmetization is designed to be modular and flexible, capturing a wide range of computations while optimizing for the patterns that arise most frequently in zkVM execution.

The frontend is based on a generalization of the *algebraic intermediate representation (AIR)* framework [Ben+18b], which specifies polynomial constraints over a witness matrix; coupled with arguments such as LogUp [Hab22], which extends AIR with logarithmic derivative-based lookups. A single AIR captures the behavior of one logical unit (e.g., the CPU, the ALU, or the memory). Since realistic computations involve multiple interacting units, we group AIRs together and define *interactions* between them —messages exchanged through shared communication channels called *buses*. This modular design has several advantages:

- **Separation of concerns**: Each AIR can be designed, implemented, and tested independently.

- **Composability**: AIRs can be reused across different computations or combined in different configurations.
- **Scalability**: Different AIRs can have different sizes, allowing resources to be allocated based on actual usage.
- **Backend independence**: The arithmetization can be paired with any compatible proving system.

The remainder of this section formalizes these concepts.

3.1. AIRs.

We generalize the standard AIR definition [Ben+18b] by augmenting it with *AIR values*: a fixed vector of field elements attached to the instance rather than indexed by row. This extension is non-standard but necessary to express global constants —such as public inputs or aggregation roots— as first-class citizens of the constraint system rather than as workarounds encoded in the trace.

Definition 1 (AIR and Witness). An *algebraic intermediate representation* (AIR) of width W, depth D, and V values is a set of *polynomials*:

$$\mathcal{A} = \{C_i\}_{i \in I} \subset \mathbb{F}\Big[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\Big]$$

where I is a finite index set. Polynomials C_i are typically known as *constraint polynomials* or simply *constraints*.

A *witness* for $\mathcal{A}$ is a pair $(\boldsymbol{T}, \boldsymbol{a})$ where $\boldsymbol{T} \in \mathbb{F}^{N \times W}$ (with N a power of two) is the *trace matrix* and $\boldsymbol{a} \in \mathbb{F}^V$ are the *AIR values*. The j-th column of $\boldsymbol{T}$, viewed as a mapping $T_j : [N] \to \mathbb{F}$, instantiates the family of shifted variables $X_j^{(0)}, ..., X_j^{(k)}$: at row r, the variable $X_j^{(k)}$ takes the value $T_j(r + k)$. The variable Y_j is instantiated by a_j, the same value at every row. The witness *satisfies* $\mathcal{A}$ if for every constraint $C \in \mathcal{A}$ and every row $r \in [N]$:

$$C(T_1(r), ..., T_W(r), ..., T_1(r + D), ..., T_W(r + D), a_1, ..., a_V) = 0$$

where row indices are taken modulo N.

Not all columns in a trace have the same lifecycle. Some are fully determined by the program before any execution begins; others depend on the specific inputs provided; and others are intermediate values the prover computes but need not reveal. Distinguishing these cases allows the proving system to avoid redundant work —a column that is the same across all executions of a program need only be committed once— and to be precise about what the verifier is entitled to see. Trace columns are therefore partitioned into three categories according to when they are determined and who can access them:

- **Fixed**: Determined at setup time, independent of any particular execution. Fixed columns are shared with the verifier as part of the AIR's description; when an AIR has fixed columns, the trace height

N is also fixed at setup. Typical uses include lookup tables and instruction encodings.

- **Cached**: Computed by the prover and committed once, then reused across multiple executions of the same program. Cached columns capture data that depends on the program but not on its inputs —such as the ROM instruction trace— avoiding redundant recomputation.
- **Witness**: Computed fresh for each execution and not revealed to the verifier. These columns carry the execution-specific state: operation values, memory accesses, and all intermediate values produced during a run.

3.2. **Interactions.**

A single AIR is self-contained: its constraint polynomials relate columns within the same trace and cannot reference columns of another AIR. Yet realistic computations are inherently cross-cutting —the CPU must read values that the memory has written, and a hash precompile must receive inputs from and return outputs to the main execution trace. To express these relationships without merging all AIRs into one monolithic constraint system, we introduce *interactions*: row-level messages that one AIR sends and another receives, mediated by a shared channel called a *bus*. An interaction specifies which values are communicated at each row and with what multiplicity; consistency across AIRs is then enforced by requiring that every bus is *balanced* —the total signed flow of messages sums to zero.

Definition 2 (Interaction). Let $\mathcal{A}$ be an AIR of width W, depth D, V values. An *interaction* of length ℓ for $\mathcal{A}$ is a tuple $(b, m, \boldsymbol{e})$ where:

- $b \in \mathbb{F}$ is a *bus identifier*.
- $m \in \mathbb{F}\left[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\right]$ is the *multiplicity* polynomial.
- $\boldsymbol{e} \in \mathbb{F}\left[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\right]^\ell$ are the *message* polynomials.

Given a witness $(\boldsymbol{T}, \boldsymbol{a})$ for the host AIR, the *evaluation* of the interaction at row r is obtained by substituting $X_j^{(k)} \leftarrow T_{j(r+k)}$ and $Y_j \leftarrow a_j$. We write $m(r)$ and $\boldsymbol{e}(r)$ for the resulting scalar and vector in $\mathbb{F}^\ell$, leaving the witness implicit when clear from context.

Each AIR can be augmented with a set of interactions. At each row, the interaction emits the message $\boldsymbol{e}(r)$ with signed multiplicity $m(r)$: a positive multiplicity means the AIR *sends* the message, a negative multiplicity means it *receives* it.

3.2.1. *Buses and Bus Balancing.*

Consider a collection of AIRs $\mathcal{A}_1, ..., \mathcal{A}_M$, each with its own set of interactions $\mathcal{I}_1, ..., \mathcal{I}_M$ and a witness $(\boldsymbol{T}_i, \boldsymbol{a}_i)$ of trace height N_i. The bus identifier b in each interaction names a shared communication channel: a *bus* is the collection of all interactions across the AIRs sharing the same bus identifier b. We denote by $\mathcal{B} = \{b : \exists i, (b, m, \boldsymbol{e}) \in \mathcal{I}_i\}$ the set of all bus identifiers in use. We assume[9] in what follows that every interaction on a given bus has a common message length ℓ.

The fundamental consistency condition is that every message sent on a bus by some AIR must be received on the same bus by another AIR with the same content: in other words, the (signed) multiset of messages emitted on a bus must sum to zero.

Definition 3 (Bus Balancing). For a bus $b \in \mathcal{B}$ and a candidate message $v \in \mathbb{F}^\ell$, define the total signed multiplicity of v on b as

$$\mathcal{M}_b(v) = \sum_{i=1}^{M} \sum_{(\tilde{b}, \tilde{m}, \tilde{\boldsymbol{e}}) \in \mathcal{I}_i} \sum_{r=1}^{N_i} \tilde{m}(r) \cdot \mathbb{1}\big[\tilde{b} = b \wedge \tilde{\boldsymbol{e}}(r) = v\big],$$

where $\mathbb{1}[\cdot]$ is the indicator function. A bus b is *balanced* if $\mathcal{M}_b(v) = 0$ for all $v \in \mathbb{F}^\ell$.

The definition is the natural multiset statement: nothing is sent that isn't received, and vice versa. As stated, however, it ranges over an exponentially large set of candidate messages and is impractical to check directly. Section 3.3 shows how *arguments* can be leveraged to compress this condition into a small set of polynomial constraints checkable by the verifier without inspecting the full witness.

Interactions and bus balancing define the communication structure, but they do not by themselves specify what *semantic* relationship is being enforced. That is the role of arguments.

3.3. **Arguments.**

Arguments are the semantic tools used to specify what consistency must hold across bus interactions: a permutation argument asserts that two multisets are equal or a lookup argument asserts containment in a table. Each argument type is realized as a pair of interactions on a shared bus — one AIR sending with positive multiplicity, another receiving with negative multiplicity— so that bus balancing becomes equivalent to the desired relationship holding. A detailed description of the specific arguments used in ZisK is provided in Appendix A.

Bus balancing is therefore the unified consistency condition underlying all arguments. However, checking it naively —summing over every row

[9]One can always take the largest message length across all interactions on a bus and pad the others accordingly.

of every AIR— requires the verifier to process the full witness, defeating succinctness. We use *LogUp* [Hab22] to reduce bus balancing to a small set of polynomial constraints that the backend can verify efficiently. The reduction proceeds in two steps: first, compress each multi-element message into a single field element; then, express the balancing sum as a polynomial identity checkable at a random location.

3.3.1. *LogUp Reduction.*

The key insight of LogUp is that multiset equality can be recast as a rational identity over a random challenge. Two multisets S and R (with multiplicities) are equal if and only if, for a *challenge* α:

$$\sum_{s \in S} \frac{1}{\alpha + s} = \sum_{r \in R} \frac{1}{\alpha + r}$$

Here, by *challenge* we refer to a random field element sent by the verifier after the prover has committed to its witness; because the prover cannot predict it before committing, the challenge breaks any attempt to craft a witness that satisfies the identity by accident. In the non-interactive setting, challenges are derived deterministically from the prover's commitments via a collision-resistant hash (Fiat-Shamir transform [FS87]), preserving the same guarantee without interaction.

To apply this idea to bus balancing, we first need to compress each multi-element message $\boldsymbol{e}(r)$ into a single field element. This compression itself uses a challenge $\gamma \in \mathbb{K}$, distinct from α but not independent of it: γ is derived deterministically from α (e.g., as a hash of α), so that fixing α determines γ as well. From the prover's and verifier's points of view, the two challenges thus form a single Fiat-Shamir-derived secret that becomes available only after the witness has been committed.

Definition 4 (Compressed Interaction Value). Given an interaction $(b, m, \boldsymbol{e})$ of length ℓ and a challenge $\gamma \in \mathbb{K}$, the *compressed value* at row r is:

$$\hat{e}(r) = b + \sum_{j=1}^{\ell} \gamma^j \cdot e_j(r)$$

The bus identifier b is included to prevent cancellation across different buses.

After compression, LogUp defines an auxiliary column to account for the sum. For each AIR $\mathcal{A}_i$ with trace of length N_i and set of interactions $\mathcal{I}_i$, the contributions of all its interactions to the bus are accumulated row by row into a *partial sum* S_i, defined by the recurrence:

$$S_i(r) = \begin{cases} \displaystyle\sum_{(b,m,\boldsymbol{e})\in\mathcal{I}_i} \frac{m(1)}{\alpha + \hat{e}(1)}, & \text{if } r = 1 \\ \displaystyle S_i(r-1) + \sum_{(b,m,\boldsymbol{e})\in\mathcal{I}_i} \frac{m(r)}{\alpha + \hat{e}(r)}, & \text{if } r > 1 \end{cases}$$

The final value $s_i = S_i(N_i)$ is the AIR's total signed contribution to the bus. Since this value is referred to throughout the rest of the paper, we give it a name.

Definition 5 (Bus Contribution). Let $(\mathcal{A}, \mathcal{I})$ be an AIR with its set of interactions, let $(\boldsymbol{T}, \boldsymbol{a})$ be a witness for $\mathcal{A}$ of trace height N, and let $\alpha \in \mathbb{K}$ be a challenge (with γ derived from α as above). The *bus contribution* of the witness is the scalar

$$\mathrm{BusCont}_\alpha(\mathcal{I}, \boldsymbol{T}, \boldsymbol{a}) := S(N) = \sum_{r=1}^{N} \sum_{(b,m,\boldsymbol{e})\in\mathcal{I}} \frac{m(r)}{\alpha + \hat{e}(r)} \in \mathbb{K},$$

i.e., the final value of the partial-sum column S defined by the recurrence above. The function is well defined whenever $\alpha + \hat{e}(r) \neq 0$ for every interaction and every row; the AIR constraints enforce this by requiring the prover to exhibit the inverse of each denominator.

Global bus balancing then reduces to a single scalar constraint:

$$\sum_{i=1}^{M} s_i = 0, \quad \text{where } s_i = \mathrm{BusCont}_\alpha(\mathcal{I}_i, \boldsymbol{T}_i, \boldsymbol{a}_i).$$

This constraint is enforced at the aggregation level: individual AIR proofs expose s_i as a public output, and the aggregation step verifies that they sum to zero (see Section 6). We make this precise as a named relation, since it is the object that the recursive verifiers of Section 6 actually check.

Definition 6 (Global Bus Balancing Relation). Fix a circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$ and a challenge $\alpha \in \mathbb{K}$. The *global bus balancing relation* is

$$\mathcal{R}^{\alpha}_{\mathrm{BUS}} = \left\{ (s_1, ..., s_M) \in \mathbb{K}^M \;\middle|\; \sum_{i=1}^{M} s_i = 0 \right\},$$

and a collection of witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}_{i=1}^{M}$ *balances every bus at* α if $(\mathrm{BusCont}_\alpha(\mathcal{I}_i, \boldsymbol{T}_i, \boldsymbol{a}_i))_{i=1}^{M} \in \mathcal{R}^{\alpha}_{\mathrm{BUS}}$.

The relation $\mathcal{R}^{\alpha}_{\mathrm{BUS}}$ is the succinct, challenge-dependent stand-in for Definition 3. The two are related by the standard LogUp soundness statement: if the witnesses do not balance some bus in the sense of Definition 3, then, for independent uniformly random $\alpha, \gamma \in \mathbb{K}$, the identity $\sum_i s_i = 0$ holds with probability at most $\frac{N_{\mathrm{tot}} + \ell \cdot N_{\mathrm{tot}}^2}{|\mathbb{K}|}$, where $N_{\mathrm{tot}} = \sum_i N_i$ is the total number

of interaction rows and ℓ the message length: the second term bounds the probability that two distinct messages collide under the compression of Definition 4, and the first bounds the roots of the resulting non-zero rational identity in α. ZisK derives γ from α rather than sampling it independently; this is recorded as a deviation from the baseline analysis in Section 9.2. The point for now is that the recursive layer checks $\mathcal{R}^{\alpha}_{\mathrm{BUS}}$, and the reduction to Definition 3 goes through the randomness of α (Section 9.5).

3.4. **Circuits.**

Similar to [Ope25], we define a *circuit* as a collection of AIRs that, together with interactions, represents a complete computation.

Definition 7 (Circuit). A *circuit* is a collection of pairs $\mathcal{C} \coloneqq \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$, where each $\mathcal{A}_i$ is an AIR with its own parameters (W_i, D_i, V_i) and each $\mathcal{I}_i$ is a set of interactions for $\mathcal{A}_i$.

A *witness* for $\mathcal{C}$ is a collection of witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}_{i=1}^{M}$ for each AIR $\mathcal{A}_i$, with $\boldsymbol{T}_i \in \mathbb{F}^{N_i \times W_i}$ and $\boldsymbol{a}_i \in \mathbb{F}^{V_i}$, where the trace heights N_i may differ across AIRs.

The fact that each AIR carries its *own* trace height N_i is a deliberate design choice and one of the main reasons for organising a computation as a circuit rather than as a single monolithic AIR. Different AIRs typically encode different units of work with different natural sizes —one AIR may scale with the length of the program being proved, another with the number of times a particular operation is invoked, and another with the size of a fixed lookup table— and there is no reason for these sizes to coincide. Forcing all AIRs to share a common height would either pad the cheap ones with empty rows or truncate the expensive ones below their needs, both of which translate directly into wasted prover work. Allowing per-AIR heights instead lets each AIR be sized to its true workload, so the prover only pays for what each unit actually does. This independence is also what later makes per-AIR mechanisms such as continuations (Section 5) possible without affecting the rest of the circuit.

Together with the bus-balancing condition introduced in Definition 3, local constraint satisfaction gives the full notion of circuit satisfiability: a circuit is satisfied when every AIR accepts its witness and every bus is balanced.

Definition 8 (Circuit Satisfiability). A circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$ is *satisfied* by witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}$ if:

1. Each witness $(\boldsymbol{T}_i, \boldsymbol{a}_i)$ satisfies its corresponding AIR $\mathcal{A}_i$.
2. Every bus $b \in \mathcal{B}$ is balanced.

4. ZisK Design

We describe the circuit architecture that constitutes the ZisK zkVM. The previous sections established the building blocks: the ISA defines what programs can run (Section 2), and the arithmetization framework defines how computations are represented as polynomial constraints (Section 3). Here we describe how ZisK instantiates that framework into a concrete system.

Definition 1 (ZisK). Given the ZisK ISA, the *ZisK zkVM* is the collection of:

1. A host execution environment capable of executing every ZisK instruction.
2. A circuit $\mathcal{C}$ (as in Section 3) that constrains a finite sequence of guest state transitions to be valid with respect to a public commitment to a program ROM and an initial guest state. The circuit exposes the final guest public outputs as a public output.
3. Methods to generate the trace matrices from the guest state transitions necessary to generate a proof for $\mathcal{C}$.

ZisK's circuit supports only a finite sequence of state transitions, whereas program execution is a priori unbounded. Fixing a single large trace height upfront is impractical: the height would have to be chosen conservatively enough to accommodate every possible program, inflating proof costs even for short executions, while still failing for programs that exceed it. *Continuations* resolve this tension by organizing and partitioning the execution trace into fixed-height segments, each proved independently. We describe how ZisK implements this mechanism in Section 5.

4.1. Architecture Overview.

The ZisK circuit is organized into *chips*, where each chip encapsulates a single AIR together with the methods to generate its trace matrix. Chips fall into two categories:

- **Instruction executor**: Owns the execution logic for a subset of the ZisK ISA. Each instruction is handled by exactly one chip, though a chip may handle multiple related instructions.
- **Table**: Provides auxiliary functionality —lookup tables, range checks, or cached computation results— shared by one or more instruction executors.

Chips communicate exclusively through *system buses*: shared channels through which chips exchange messages. The set of system buses used in ZisK is summarized in Table 2. While there are globally shared buses, each chip may expose interfaces for communicating with other particular chips. This modular design means that new instructions can be supported by introducing a new chip and connecting it to the appropriate buses, without modifying any existing chip.

Bus	Purpose
Operation Bus	Routes operations requiring dedicated constraints from some chip to other specialized chips
ROM Bus	Verifies each instruction fetch against the committed program ROM
Memory Bus	Enforces read/write consistency across all memory accesses
Continuation Buses	Carries chip state across segment boundaries
Table Buses	Connects instruction executor chips to shared lookup tables

TABLE 2. Bus system in ZisK. Each bus serves a specific purpose in the architecture, enabling modular communication between chips and supporting the overall proof structure.

An instruction executor may delegate part of its computation to other chips via a bus, which may in turn query a table chip, and so on. This creates a layered dependency structure, illustrated in Figure 1: each chip proves only its own local constraints, and bus balancing composes the correctness arguments upward into a global proof.

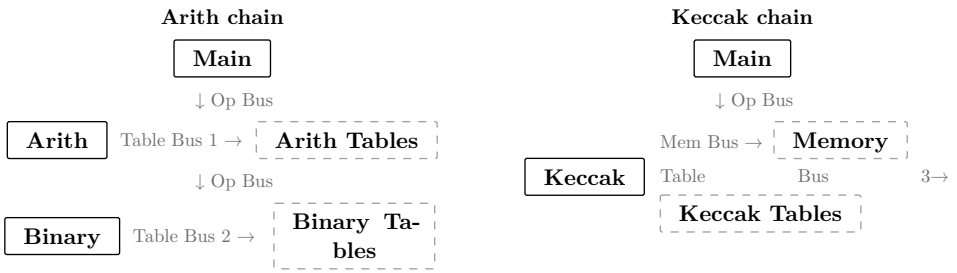


FIGURE 1. Two example chip dependency chains in ZisK. Solid borders denote instruction executor chips; dashed borders denote table chips. **Left**: the Main chip dispatches an arithmetic operation to Arith via the Operation Bus; Arith queries its own tables and further delegates to Binary, which queries its own tables. **Right**: Main dispatches a cryptographic operation to Keccak; the precompile simultaneously reads its inputs and writes its outputs via the Memory Bus and queries its own tables. In both cases each chip proves only its own local constraints, and bus balancing composes the arguments into a global proof.

4.1.1. *Chip Inventory.*

Table 3 enumerates every AIR of the ZisK circuit $\mathcal{C}$ at v0.16.1, with its trace height N_i and the category it belongs to. This is the concrete instantiation of the abstract circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$ of Section 3: $M = 35$, of which 32 are instruction-executor or memory AIRs and 3 are table AIRs. The recursion circuits of Section 6 are not part of $\mathcal{C}$; they are listed in the topology of Section 9.1. Several chips are realised by more than one AIR (the Memory chip by three region AIRs, MemAlign by four, DMA by twelve); conversely, every AIR belongs to exactly one chip, so "chip" is a grouping of AIRs for exposition and the formal objects are the AIRs.

AIR	$\log_2 N_i$	Category	Proves
Main	22	executor (spine)	one instruction step per row: fetch, operand resolution, dispatch, store, control flow, register argument
Rom	22	table (cached)	the program P as a lookup table on the ROM Bus; cached columns committed per program
Mem	22	memory	sorted access trace of the writable RAM region
RomData	21	memory	sorted access trace of the read-only data region
InputData	21	memory	sorted access trace of the input-data region (free values)
MemAlign	21	memory helper	sub-word and unaligned accesses as sequences of aligned ones
MemAlignByte, MemAlignReadByte, MemAlignWriteByte	22	memory helper	single-byte accesses (read+write, read-only, write-only variants)
Arith	21	executor	64/32-bit multiplication, division and remainder
Binary	22	executor	byte-sliced comparisons, min/max, subtraction, bitwise logic
BinaryAdd	22	executor	64-bit addition
BinaryExtension	22	executor	shifts, rotations, sign extension
Add256	20	precompile	256-bit addition with carry
ArithEq	20	precompile	256-bit modular arithmetic, secp256k1/secp256r1/BN254 curve and $\mathbb{F}_{p^2}$ operations
ArithEq384	20	precompile	384-bit modular arithmetic and BLS12-381 curve and $\mathbb{F}_{p^2}$ operations
Keccakf	17	precompile	Keccak-f[1600] permutation
Sha256f	18	precompile	SHA-256 compression function
Poseidon2	17	precompile	Poseidon2 permutation of width 16
Blake2br	18	precompile	BLAKE2b round function
Dma, DmaMemCpy, DmaInputCpy	21	precompile	DMA front-ends: decode a bulk copy/compare/set into phases
Dma64Aligned and 4 variants	21	precompile	aligned bulk copy, compare and set loops
DmaUnaligned	21	precompile	unaligned bulk copy and compare
DmaPrePost, DmaPrePostMemCpy, DmaPrePostInputCpy	21	precompile	head and tail partial words of a DMA operation
VirtualTable0, VirtualTable1	20	table	packed range-checked lookup tables ($[0, 2^{20})$, $[0, 2^{22})$, $[0, 2^{24})$), bitwise and signed-extension tables, MemAlign program, DMA tables, frequent-operation caches, dual-byte ranges

TABLE 5. The $M = 35$ AIRs of ZisK v0.16.1. Instances of each AIR are created on demand by trace splitting (Section 5); an execution uses as many instances of each AIR as its workload requires, and none of an AIR whose operation never occurs.

Now we delve into the details of the design of the ZisK chips (Section 4.2) and bus system (Section 4.4).

4.2. **Chip Design.**

We now turn to the chips themselves: the AIRs that, together with the bus system, implement the ZisK zkVM. The presentation roughly follows the dependency structure of Figure 1, starting from the central spine and moving outwards. We begin with the *Main* chip (Section 4.2.1), which materializes one row per instruction step and orchestrates the rest of the system: it fetches instructions, resolves operands, dispatches operations to specialized sub-chips, and writes back results. We then describe the *Memory* chip (Section 4.2.2), which centralizes the memory consistency argument that almost every other chip ultimately relies on. The remaining categories cover what the Main delegates: *base-operation* chips (Section 4.2.3) handle classes of simple operations —binary, arithmetic, comparisons— through dedicated AIRs and lookup tables; *precompiles* (Section 4.2.4) handle expensive or memory-intensive primitives such as elliptic-curve arithmetic, hash functions, and bulk transfers, each as an independent instruction executor with its own internal layout; and *frequent-operation caches* (Section 4.2.5) commit pre-computed results for the most commonly queried inputs of base operations, replacing constraint checking with a direct lookup wherever possible.

4.2.1. *The Main.*

ZisK uses a *Main* chip as the central execution spine. Each row of the Main AIR corresponds to one instruction step and its trace materializes the full execution transcript. Recalling the notation from Section 2.2.2, the Main chip is responsible for:

1. **Instruction fetch**: At each step, the Main chip reads the instruction at the current program counter from the program ROM via the ROM Bus (Section 4.4.2).
2. **Operand resolution**: The input operands a and b are resolved according to their source modes (immediate, memory, or indirect), with memory reads constrained through the Memory Bus.
3. **Operation dispatch**: Operations that are simple enough are computed directly in the Main chip (e.g., no-ops, jumps, and control flow). Operations requiring dedicated constraint systems — binary operations, multiplications, cryptographic primitives— are *delegated*: the Main chip sends the inputs and receives back the result (c, flag) via the Operation Bus, and a specialized sub-chip proves correctness.
4. **State update**: The output c is written to its destination via the Memory Bus or directly in the Main trace.

4.2.1.1. *Registers.*

Although the ZisK ISA addresses the 32 general-purpose registers within the same numerical address space as data memory (Section 2.2.6), ZisK does *not* route register accesses through the Memory chip. Forcing every register read and write to traverse the sorted memory trace would inflate it — registers are by far the most frequently accessed locations in any execution — and would couple the cost of register access to the cost of generic RAM. Instead, the register region is carved out of the Memory chip's domain and handled by the Main chip with a *linked-step argument* on the Memory Bus.

Concretely, every register access at memory step t_{mem} (Section 4.2.1.2) to register $i \in [1, 31]$ with value v is accompanied in the Main trace by the memory step t_{prev} of the previous access to the same register. The Main chip *receives* the message $\big(\texttt{REG}, i, t_{\text{prev}}, 8, v_{\text{prev}}\big)$ —asserting that the previous access exists and left value v_{prev}— and *sends* $(\texttt{REG}, i, t_{\text{mem}}, 8, v)$, where $v_{\text{prev}} = v$ for a read and v_{prev} is a free witness column for a write. A range check $0 \le t_{\text{mem}} - t_{\text{prev}} - 1 < 2^{24}$ forces the chain of accesses to each register to be strictly increasing in time. The chain is opened once, globally, by a receive of $(\texttt{REG}, i, 0, 8, 0)$ for every i (registers start at zero), and it is closed by the last segment; at every segment boundary the Main chip reloads all 31 registers (Section 5.3), which is what keeps the 2^{24} window sufficient. The register messages use the opcode `REG` and the bare index i as address, so they can never be confused with a RAM message. Section 4.2.2.1 states the relation this argument enforces and its refinement to the ISA view in which registers are memory cells.

4.2.1.2. *Timestamps.*

The circuit arithmetization does not inherently represent temporal order. On the other hand, the execution of instructions is inherently sequential. To encode it, ZisK discretizes time by attaching a *timestamp* to every memory access, so that the memory consistency argument can verify that reads always observe the most recent write to each address.

Time is organized at two granularities. A *main step* $t \in [0, 2^{36})$ indexes one full instruction execution by the Main; it is the global step counter of the execution model, reconstructed inside a segment as $t = \text{seg} \cdot N + r$ from the segment index seg (an AIR value, range-checked) and the row r. A single main step, however, is not atomic from the memory's perspective: operand resolution requires reads, storing the result requires a write, and a dispatched precompile may independently access memory to read its inputs and write its outputs. To order all of these accesses, each main step t is subdivided into four *memory steps*

$$t_{\text{mem}} = 1 + 4 \cdot t + s, \quad s \in \{0, 1, 2, 3\},$$

with the following fixed slot assignment: $s = 0$ for the read of operand a, $s = 1$ for the read of operand b, $s = 2$ for the store of c and for the reads of a precompile dispatched at step t, and $s = 3$ for the writes of such a precompile, for the write-back that MemAlign performs on behalf of a slot-2 store, and for the end-of-segment register reload. Memory step 0 is reserved for the global initialisation of the registers. Since $t < 2^{36}$, memory steps fit in 38 bits. The slot is the only ordering device: the address accessed at each slot is unconstrained and varies freely from instruction to instruction and from chip to chip, but an operation dispatched at step t can only stamp its accesses with the memory steps of step t, because the Main chip places t on the Operation Bus for every precompile call (Section 4.4.1) and the precompile chip derives its memory steps from that value.

4.2.2. *Memory.*

The *Memory* chip centralizes all memory access verification. It is realised by three AIRs of the same template, one per address region (Table 3): Mem for writable RAM, RomData for the read-only data, and InputData for the input-data region. Each AIR collects every memory operation submitted on the Memory Bus for its region, verifies their consistency, and rejects any execution in which a memory access violates the region's rules.

To enforce correctness, all submitted operations are sorted globally by address, then by memory step within the same address. This reduces the global consistency requirement —that every read observes the most recent prior write— to a sequence of local, row-to-row checks. We now state precisely what the AIR enforces; the constraint system is the one of `mem.pil` at v0.16.1, and the description below is complete with respect to the soundness-relevant constraints.

4.2.2.1. *Memory Consistency Relation.*

A row of a Memory AIR is a tuple $(\rho, \text{addr}, t_{\text{mem}}, \text{wr}, \text{sel}, v)$ with $\rho \in \{0, 1\}$ the *address-change* flag, $\text{addr} \in [0, 2^{29})$ a word address, t_{mem} a memory step, $\text{wr} \in \{0, 1\}$ the write flag, $\text{sel} \in \{0, 1\}$ the *bus selector*, and $v = (v_0, v_1) \in [0, 2^{32})^2$ the value as two limbs. Rows with $\text{sel} = 1$ emit the message $(\text{op}, 8 \cdot \text{addr}, t_{\text{mem}}, 8, v_0, v_1)$ on the Memory Bus with $\text{op} = \texttt{STORE}$ if $\text{wr} = 1$ and `LOAD` otherwise, as a *prove* (positive multiplicity); rows with $\text{sel} = 0$ emit nothing and exist only to bridge address gaps or to pad the trace. The Memory Bus is a *permutation* argument (Appendix A): the multiset of messages proved by the Memory AIRs must equal the multiset of messages assumed by all requesters.

Definition 2 (Sorted Access Trace). Fix a region with word-address interval $[\text{base}, \text{end}]$. A sequence of rows $(\rho_r, \text{addr}_r, t_r, \text{wr}_r, \text{sel}_r, v_r)_{r\in[R]}$, together with a *previous row* $(\text{addr}_0, t_0, v_0)$ supplied by the continuation bus (Section 5.3), is a *sorted access trace* of the region if for every $r \in [R]$:

1. (*Region*) $\text{base} \le \text{addr}_1$ and $\text{addr}_R \le \text{end}$ (each enforced by a 32-bit range check on the distance).
2. (*Address order*) $\rho_r = 1 \Rightarrow \text{addr}_r > \text{addr}_{r-1}$, and $\rho_r = 0 \Rightarrow \text{addr}_r = \text{addr}_{r-1}$.
3. (*Time order*) $\rho_r = 0 \Rightarrow t_r - t_{r-1} \ge \text{wr}_r$, i.e. strictly increasing steps for writes and non-decreasing steps for reads, so that several reads of the same word at the same memory step are allowed.
4. (*Bounded increments*) the quantity $\rho_r(\text{addr}_r - \text{addr}_{r-1}) + (1 - \rho_r)(t_r - t_{r-1} + 1 - \text{wr}_r) - 1$ lies in $[0, 2^{38})$, witnessed by a 22-bit and a 16-bit range-checked limb.
5. (*No silent writes*) $\text{wr}_r = 1 \Rightarrow \text{sel}_r = 1$.
6. (*Read-after-write*) $\rho_r = 0 \wedge \text{wr}_r = 0 \Rightarrow v_r = v_{r-1}$.
7. (*Region policy*) for the writable RAM region, $\rho_r = 1 \wedge \text{wr}_r = 0 \Rightarrow v_r = 0$ (an address that is read before ever being written reads as zero). For the read-only data region, $\text{wr}_r = \rho_r \cdot \text{sel}_r$: the first access to each address is a write and all later accesses are reads. For the input-data region, $\text{wr}_r = 0$ and, except at the free-input port (the region's base address), $\rho_r = 0 \Rightarrow v_r = v_{r-1}$; at the free-input port the value is unconstrained on every row.

The relation $\mathcal{R}^{\text{region}}_{\text{MEM}}$ consists of all $(x_{\text{mem}}; w_{\text{mem}})$ where w_{mem} is such a trace and x_{mem} is the pair of boundary rows carried on the continuation bus.

The witness-side facts that make the definition sound are recorded here because the soundness sketch relies on them. Padding rows repeat the last row's address, step and value with $\text{sel} = \text{wr} = 0$, which satisfies every constraint and emits nothing. Because increments are bounded by 2^{38} on the RAM region, whose address space is 2^{29} words and whose time span is 2^{38} memory steps, no address or time gap ever requires a bridging row there; the read-only and input-data regions use a 2^{24} address-increment bound and insert bridging rows with $\text{sel} = 0$ where an address gap exceeds it. The selectors are constrained to be binary, so a row cannot emit a message with multiplicity greater than one.

What the sorted access trace guarantees is the semantic memory relation of the execution model. Say that a multiset O of memory operations $(\text{op}, x, t_{\text{mem}}, w, v)$ is *consistent with an initial memory* mem_0 if, processing the operations in order of t_{mem}, every `LOAD` returns the value most recently stored at its address, or the initial value if none.

Lemma 3 (Memory Argument). Let O be the multiset of aligned memory messages assumed by the requester AIRs of a satisfying circuit witness, restricted to a region, and suppose the Memory Bus balances. Then the region's Memory AIR instances jointly form a sorted access trace whose $\mathrm{sel} = 1$ rows are exactly O, and O is consistent with the region's initial memory: zero for RAM, the values of the first (write) access to each address for the read-only data region, and an arbitrary per-address constant for the input-data region. In particular every read of the RAM region observes the most recent prior write to its address, and the input-data region behaves as a read-only memory with prover-chosen contents whose values, once fixed by the first read, are the same for every subsequent read.

The proof is the standard one for sorted memory-checking [Blu+91]: address and time order make the trace a concatenation of per-address blocks ordered in time; the permutation argument makes the blocks contain exactly the requested operations (with duplicates counted); read-after-write and the region policy pin the value of every read. The lemma is stated as a proof obligation in Section 9.5 (it is a property of the constraint system, not of the cryptography). Two refinements connect the lemma to the ISA view of memory in Section 2.3: the *register refinement*, which shows that the linked-step argument of Section 4.2.1.1 enforces the same consistency property for the 31 register cells, and the *MemAlign refinement*, which shows that the sub-word and unaligned operations proved by MemAlign are equivalent to the ISA's byte-granular reads and writes (Section 4.2.2.4).

Remark (Binding of the read-only data region). *Lemma 3 makes the contents of the read-only data region equal to the values of the* first *access to each address, which is a write requested on the Memory Bus. Whether those values equal the constants* $\mathrm{ROMDATA}_P$ *of the program binary therefore depends on which chip requests the initialising writes and on how that chip is bound to the program commitment* $\mathrm{root}_{\mathrm{ROM}}$*. In the current ZisK sources the ROM AIR emits the initialising writes from its committed cached columns, which binds them to* $\mathrm{root}_{\mathrm{ROM}}$*; the corresponding proof obligation for v0.16.1 is listed explicitly in Section 9.5.*

4.2.2.2. *Memory Access Paths.*

The access path for a memory operation depends on its width and alignment, as shown in Figure 2:

- **Aligned 8-byte access**: The requesting chip places the operation directly on the Memory Bus. The Memory chip processes it without further decomposition.
- **Sub-word or unaligned access** (1, 2, or 4 bytes): The requesting chip routes the operation to the *MemAlign* chip instead. MemAlign identifies whether the access falls within a single aligned 8-byte

word or spans two consecutive words, decomposes it accordingly, and issues one or two aligned 8-byte operations to the Memory Bus.

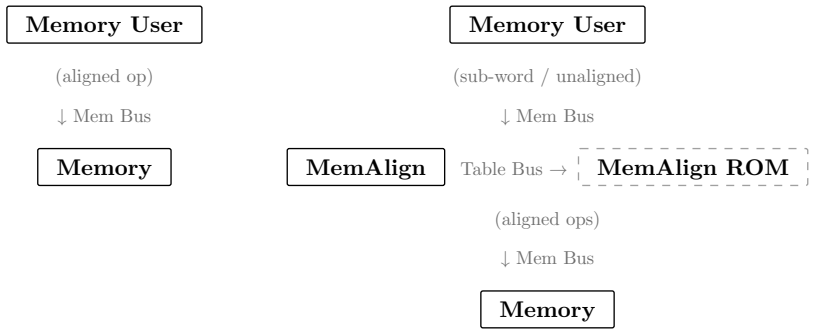


FIGURE 2. Memory access paths. Aligned 8-byte operations go directly to Memory; sub-word and unaligned operations are first decomposed by MemAlign into aligned 8-byte operations.

4.2.2.3. *MemAlign.*

The MemAlign chip models a small processor (the MemAlign ROM) with eight byte-wide registers that hold one aligned 8-byte word, and executes one of four fixed subprograms depending on whether the access is a read or write and whether it fits within one aligned word or spans two:

1. **Read within one word** [One read]: MemAlign performs one aligned read to load the surrounding 8-byte word into its registers, then extracts the requested bytes and returns the value.
2. **Write within one word** [One read, One write]: MemAlign reads the surrounding word, patches the relevant byte positions with the new value, writes the modified word back, and confirms the result.
3. **Read spanning two words** [Two reads]: The requested bytes straddle an 8-byte boundary. MemAlign reads both words, combines the relevant bytes from each, and returns the assembled value.
4. **Write spanning two words** [Two reads, Two writes]: MemAlign reads both words, patches the relevant bytes in each, and writes both modified words back.

In every case, MemAlign issues only aligned 8-byte operations to the Memory Bus. This design allows MemAlign to support all possible sub-word and unaligned accesses with a small, fixed set of constraints, without needing to know the specific access patterns in advance.

4.2.2.4. *MemAlign Refinement.*

MemAlign receives from the requester the unaligned message $(\mathrm{op}, x, t_{\mathrm{mem}}, w, v)$ with $w \in \{1, 2, 4\}$ or $8 \nmid x$, and assumes from the Memory chip one or two aligned messages: the reads $(\texttt{LOAD}, 8\lfloor x/8 \rfloor, t_{\mathrm{mem}}, 8, u_0)$ and, if the access spans two words, $(\texttt{LOAD}, 8\lfloor x/8 \rfloor + 8, t_{\mathrm{mem}}, 8, u_1)$; for a write,

additionally the stores of the patched words at memory step $t_{\text{mem}} + 1$. The fixed MemAlign program table (a lookup on its own table bus) forces the byte selection and patching to be the little-endian extraction and replacement of bytes $x \bmod 8, ..., x \bmod 8 + w - 1$ of the concatenation $u_0 \parallel u_1$. The stamping of the write-back at $t_{\text{mem}} + 1$ is what preserves the order of the read-modify-write pair inside the requester's slot, and it is the reason a store slot is followed by a reserved slot in Section 4.2.1.2.

Lemma 4 (MemAlign Refinement). In a satisfying circuit witness with balanced buses, every unaligned or sub-word message assumed by a requester is realised by aligned messages on the Memory Bus whose net effect on the byte-granular memory function of Section 2.3 is exactly $\text{mem}[x]_w$ for a load and $\text{mem}[x]_w \leftarrow v$ for a store, and no other byte is modified.

4.2.3. *Base Operations.*

The *Base Operations* chips prove the base instruction set operations — bitwise logic, comparisons, shifts, sign extensions, and integer multiplication and division— delegated from the Main chip via the Operation Bus.

These chips share a common structural profile that distinguishes them from the Main chip and from precompiles. First, they have no interaction with the Memory Bus: their inputs and outputs flow entirely through the Operation Bus, so they neither read from nor write to the memory state. Second, they carry no state across segment boundaries and therefore use no continuation bus; each segment's proof is self-contained. These two properties mean that Base Operations chips are the simplest kind of instruction executor in ZisK.

Base Operations chips together with the Main chip and Memory chip form the core execution engine of ZisK, responsible for executing every instruction in the ZisK ISA. These chips are not designed for any particular class of operations; rather, they are designed to be flexible enough to support the full range of operations in the ISA, and their constraints are structured to allow efficient delegation of complex operations to specialized sub-chips.

Precompiles, on the other hand, are designed to handle complex operations that would be expensive to constrain step-by-step in the Main chip.

4.2.4. *Precompiles.*

Precompiles handle classes of operations —elliptic curve arithmetic, cryptographic hash functions, bulk memory transfers— whose decomposition into base operations would be inefficient. Each precompile is an independent instruction executor chip that owns the full proof obligation for its operation.

What distinguishes precompiles from basic operation chips is the complexity of their interface with the rest of the system. A precompile typically reads its inputs and writes its outputs directly from the memory state through the Memory Bus, rather than receiving them as simple 64-bit operands on the Operation Bus. Its computation may span many internal rows —for example, a hash permutation iterates over multiple rounds— so its trace height is sized to the operation's intrinsic complexity, independently of the Main chip. Some precompiles (such as DMA) even carry their own internal state across segment boundaries via a continuation bus.

At the ISA level, invoking a precompile follows a calling convention: the caller issues a sequence of parameter-passing instructions, a call instruction, and result-retrieval instructions. These steps appear in the Main chip trace as bookkeeping steps that set up and tear down the call, but carry no proof obligation themselves. Adding a new precompile requires only defining a new chip and wiring it to the appropriate buses, with no changes to any existing chip.

4.2.5. *Frequent-Operation Caches.*

Basic operation chips carry a fixed per-invocation overhead: each chip handles a whole family of operations, and a single query pays for the constraints that distinguish all of them, regardless of which one is actually being proved. Across a typical program execution, however, only a small fraction of all possible input combinations actually appear, and those that do appear tend to repeat. ZisK exploits this with *frequent-operation caches*: for binary and arithmetic operations, results for the most commonly queried input pairs are pre-computed and committed once at setup time.

At proof time, sub-chips resolve queries directly from the cache for inputs it covers, and fall back to constraint checking for the remainder. This significantly reduces witness size and proving time for programs with typical workloads.

4.3. **Word Encoding over Goldilocks.**

The ZisK ISA operates on 64-bit unsigned integers, while the arithmetization works over the Goldilocks field with $p = 2^{64} - 2^{32} + 1 < 2^{64}$. A 64-bit word therefore does not fit canonically into one field element, and integer overflow must occur at 2^{64}, not at p. ZisK resolves this with a uniform encoding, which every bus schema in Section 4.4 and every trace column holding an ISA value adheres to.

- **Canonical limbs.** A 64-bit value v is represented as two field elements (v_0, v_1) with $v = v_0 + 2^{32}v_1$ and $v_0, v_1 \in [0, 2^{32})$. Since $2^{32} < p$, each limb has a unique canonical representative, and the pair determines v uniquely. The Main chip, the Operation Bus, the Memory Bus and the Memory AIRs all use this two-limb form; the

`_w` opcodes zero the high limb of their operands on the Operation Bus.

- **Range constraints.** A limb column is a valid 32-bit limb only if constrained to $[0, 2^{32})$. In ZisK this is enforced at the point where a value is *produced*: the executor chips decompose their outputs further (bytes in Binary and BinaryExtension, 16-bit chunks in BinaryAdd, Arith and the 256/384-bit precompiles) and range-check the pieces via the table buses, and reconstruct the limbs as $v_0 = \sum_k 2^{8k} b_k$ or $\sum_k 2^{16k} h_k$. A value that is only *consumed* (e.g. an operand received by the Main from a register) inherits its range from the message that produced it, since the bus transports the exact limbs.
- **Overflow at 2^{64}.** Arithmetic operations are verified on the decomposed form with explicit carries: addition as $a_0 + b_0 = c_0 + 2^{32}\kappa_0$, $a_1 + b_1 + \kappa_0 = c_1 + 2^{32}\kappa_1$ with $\kappa_i \in \{0, 1\}$ and κ_1 discarded; multiplication in Arith as the integer identity $a \cdot b + c = 2^{64} d_{\text{hi}} + d_{\text{lo}}$ over 16-bit chunks with range-checked carries. None of these identities can wrap modulo p, because every term is bounded far below p by its range constraints.
- **Addresses and steps.** Addresses are 32-bit and carried as a single field element; memory steps are 38-bit and carried as a single field element; both are bounded by range checks or by the increment argument of Definition 2. The Memory chip's word address is the byte address divided by 8; the bus carries the byte address.

The consequence for the security analysis is that the map from a satisfying trace to ISA values is well defined and injective on every column that the relations of Section 7 read, so an extractor can decode 64-bit words from field elements without ambiguity. The obligation that every limb column consumed by a relation is actually range-constrained is recorded in Section 9.5.

4.4. **System Buses.**

ZisK uses a bus system —recall Table 2 for an overview— to constrain valid state transitions during program execution. Every bus is an instance of the bus-balancing primitive of Section 3 with a fixed *message schema* and a fixed *semantic relation* that balancing enforces; we give both for each bus. All 64-bit values use the two-limb encoding of Section 4.3. The continuation buses are described in Section 5.3.

4.4.1. *Operation Bus.*

The *Operation Bus* (identifier 5000) routes operations from the Main chip to the executor and precompile chips. Messages have the form

$$(\text{op}, a_0, a_1, b_0, b_1, c_0, c_1, \text{flag}, t, x, \text{extra}),$$

where op $\in [0, 2^8)$ identifies the operation, $(a_0, a_1), (b_0, b_1), (c_0, c_1)$ are the operands and result, flag $\in \{0, 1\}$, t is the main step, x is the extended argument (the `j1` field of a precompile instruction) and extra is a chip-specific field used only by the DMA chips. The Main chip *sends* one message per row with is_external_op $= 1$, setting $t = x = 0$ unless the instruction is a precompile call, and zeroing a_1, b_1 for `_w` opcodes; the frequent-operation caches *receive* messages of the form $(\text{op}, a_0, a_1, b_0, b_1, c_0, c_1, \text{flag})$ (trailing zeros are implicit); every executor and precompile chip *receives* the messages for its opcodes; and the global public-output binding of Section 7 *receives* the 32 `pubout` messages. Executor chips whose rows are not all used cancel their padding rows with a degree-zero term (Section 5.3).

Definition 5 (Operation Bus Relation). A message $(\text{op}, a, b, c, \text{flag}, t, x, \text{extra})$ received by a non-precompile chip is *valid* if $(c, \text{flag}) = \text{op}(a, b)$ according to Appendix B. A message received by a precompile chip is valid if $(c, \text{flag}) = \text{op}(a, b)$ and the chip's memory messages (Section 4.4.3) at memory steps $1 + 4t + 2$ (reads) and $1 + 4t + 3$ (writes) realise the memory effect Eff_{op} of Section 2.3. Balancing of the Operation Bus in a satisfying circuit witness implies that every message sent by the Main is valid.

The implication holds because each receiving AIR's constraints accept only valid messages for the opcodes it handles, the frequent-operation tables are fixed columns containing only valid triples, and the multiset equality of the lookup argument means every sent message equals some received one. The only sender is the Main chip (and, for the `pubout` opcode, the global binding), so the relation is a property of the Main's rows.

4.4.2. *ROM Bus.*

The *ROM Bus* (identifier 7890) ensures correctness of instruction fetching. Messages have the form

$$(\text{pc}, \text{a_off}, \text{a_imm1}, \text{b_off}, \text{b_imm1}, \text{ind_w}, \text{op}, \text{store_off}, \text{j1}, \text{j2}, \text{flags}),$$

where the fields are those of Section 2.2.2 and flags packs the boolean fields as

$$\begin{aligned}\text{flags} = 1 + 2 \cdot [\texttt{a_src} = \texttt{IMM}] + 2^2[\texttt{a_src} = \texttt{MEM}] + 2^3\ \text{is_precompiled} + 2^4[\texttt{b_src} = \texttt{IMM}] + 2^5[\texttt{b_src} = \texttt{MEM}] + 2^6\ \text{is_external_op} \\ + 2^8[\texttt{store} = \texttt{MEM}] + 2^9[\texttt{store} = \texttt{IND}] + 2^{10}\ \text{set_pc} + 2^{11}\ \text{m32} + 2^{12}[\texttt{b_src} = \texttt{IND}] + 2^{13}[\texttt{a_src} = \texttt{REG}] + 2^{14}[\texttt{b_src} = \texttt{REG}] + 2^{15}[\texttt{s}\end{aligned}$$

The constant 1 makes every genuine instruction distinct from the all-zero padding rows of the table. The Main chip *sends* one message per row, formed from its decoded instruction columns; the ROM chip *receives* them with a free multiplicity column against its cached columns, which hold the table P. Since the ROM chip's columns are committed once per program and bound to the program commitment root_{ROM} (Section 8.4), balancing gives:

Definition 6 (ROM Bus Relation). Balancing of the ROM Bus in a satisfying circuit witness implies that at every Main row the decoded instruction equals $P[\mathrm{pc}]$, where P is the program committed to by $\mathrm{root}_{\mathrm{ROM}}$; i.e. $\iota_t = \mathrm{Fetch}_P(S_t)$ for every step t.

4.4.3. *Memory Bus.*

The *Memory Bus* (identifier 10) is the channel through which every chip that reads from or writes to an address space submits its memory operations. Messages have the form

$$(\mathrm{op}_{\mathrm{mem}}, \mathrm{addr}, t_{\mathrm{mem}}, \mathrm{width}, v_0, v_1),$$

where $\mathrm{op}_{\mathrm{mem}} \in \{\texttt{LOAD} = 1, \texttt{STORE} = 2, \texttt{REG} = 3\}$, $\mathrm{addr} \in [0, 2^{32})$ is the byte address (or the register index for `REG` messages), t_{mem} is the memory step (Section 4.2.1.2), $\mathrm{width} \in \{1, 2, 4, 8\}$, and (v_0, v_1) is the value. Requesters —Main, MemAlign, the precompiles— *send* their operations; the three Memory AIRs *receive* aligned ones; MemAlign receives unaligned ones and sends the aligned ones it decomposes them into; and the Main both sends and receives the `REG` messages of its linked-step argument. The bus is a permutation argument, and its semantic relation is Lemma 3 together with its two refinements: balancing implies that the requested operations, taken over the whole execution, are consistent with the initial memory mem_0 of Section 2.3.

4.4.4. *Table Buses.*

The remaining buses connect executor chips to fixed tables. Each is a lookup argument (Appendix A) whose receiving side is a table AIR with fixed columns and a free multiplicity column, so its semantic relation is simply membership of the sent message in the table. Table 4 lists them. The range-check buses are the specialisation of Appendix A; all of them are hosted in the SpecifiedRanges AIR, and the two dual-byte tables in the VirtualTable AIRs.

Table	Id	Message schema and meaning	Senders
BinaryTable	125	$(\text{pos}, \text{op}, a, b, \text{cin}, c, \text{flags})$: one byte of a byte-sliced binary operation with carry in and result flags	Binary
BinaryExtensionTable	124	$(\text{op}, \text{off} \mid \text{flags}, a, b, c_0, c_1)$: one byte of a shift, rotation or sign extension	BinaryExtension
ArithTable	331	$(\text{op}, \text{flags}, \text{range}_{\text{ab}}, \text{range}_{\text{cd}})$: operation decoding and range selectors for Arith	Arith
ArithRangeTable	330	$(\text{range_id}, v)$: 16-bit chunk and signed-carry ranges for Arith	Arith
MemAlignRom	133	$(\text{pc}, \Delta\ \text{pc}, \Delta\ \text{addr}, \text{off}, w, \text{flags})$: the MemAlign micro-program	MemAlign
ArithEqLtTable	5002	(lt, Δ): chunk-wise comparison for modular reduction	ArithEq, ArithEq384
KeccakfTable	126	bit-sliced Keccak round tables	Keccakf
FrequentOps (3 tables)	5010–5012	$(\text{op}, a, b, c, \text{flag})$ on the Operation Bus: cached results of frequent arithmetic, binary and shift operations	(receive from Main)
DmaRom, DmaPrePostTable, DmaByteCmpTable	8001–8003	DMA micro-program, partial-word and byte-comparison tables	Dma chips
DualByte, DualRange7Bit	88, 77	(x, y) with $x, y \in [0, 2^8)$ resp. $[0, 2^7)$: two range checks per lookup	MemAlign, Dma chips
Range checks	≥ 100	(x) with $x \in [\text{lo}, \text{hi}]$ for each declared range	all chips

TABLE 4. Table buses of ZisK v0.16.1. Every table is a fixed column set committed at setup (Section 8.4); the DMA chips additionally use an inter-chip permutation bus (identifier 8000) to hand in-flight operations between their front-end and worker AIRs.

5. CONTINUATIONS

A single chip proof covers a trace of fixed height N. Real programs run for an a priori unbounded number of steps, so a single proof cannot certify an arbitrary execution. *Continuations* are the mechanism by which a zkVM extends beyond this bound: the execution is divided into *segments*, each covered by an independent proof, and the proofs are linked into a single argument for the full run.

5.1. **The Standard Approach to Continuations.**

The most common continuation strategy splits execution into *checkpointed* segments. Execution runs for N steps, halts, and the prover generates a proof for those N steps; the machine state at the halt —the

checkpoint— is saved. A fresh execution then resumes from that checkpoint and runs for another N steps, and so on. Each segment proof certifies one N-step block, and consecutive proofs are chained by having each segment verify the preceding checkpoint as part of its own public inputs.

This approach carries a significant overhead: *padding*. If a chip performs only $k \ll N$ operations in a given segment, its $N - k$ remaining rows must be filled with dummy entries that satisfy the constraints but carry no useful computation. In a multi-chip system, this cost compounds across every chip in every segment: different chips have wildly different utilization across segments, yet each padded row still enters the polynomial commitment and constraint evaluation. The result is a large blowup in prover time and proof size, which grows with the number of segments and the number of chips.

A solution at the cryptographic layer is the *jagged polynomial commitment scheme (Jagged PCS)* [Hem+25]. This protocol commits to the full computation trace as a single polynomial over a jagged matrix —where each column corresponds to a chip and can have a different height— and lets the verifier emulate access to the individual per-chip polynomials. The prover pays proving cost proportional to the actual trace area rather than the padded one. The jagged PCS is, however, inherently multilinear: it is built on multilinear polynomial commitments and can be incorporated efficiently only into proving systems whose arithmetization is multilinear-based.

Rather than developing new cryptographic infrastructure and tying the solution to a specific class of proving system, ZisK addresses the problem at the system design level, described next.

5.2. **ZisK's Approach: Trace Splitting.**

ZisK decouples segmentation from execution and lets chips operate on demand. The program runs to completion in the host execution environment in a single pass, producing the full execution trace. Each chip records only the rows for instructions it handles: the Memory chip records only memory accesses, Keccak only hash operations, and so on. Chips that are rarely invoked produce short traces; chips that are heavily used produce long ones.

The prover then partitions each chip's trace independently into consecutive segments of that chip's fixed height N_i. Padding is limited to the final segment of each chip, where the last partial slice is padded to N_i rows; all earlier segments are filled to capacity. This stands in contrast to the standard approach, where every chip is padded to its height in every execution segment regardless of how many operations it actually performed. Figure 3 illustrates this layout.

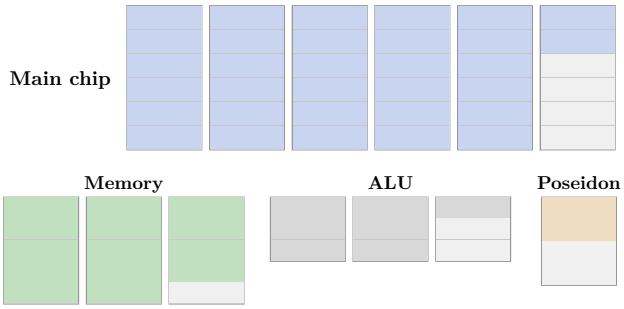


FIGURE 3. Padding layout for ZisK's trace-splitting approach. Chips define their own segment height N_i. Colored rows hold actual computation; gray rows are padding. Only the final segment of each chip requires padding.

5.2.1. *Segments as Formal Objects.*

We now make the segmentation precise. Fix a successful execution $S_0, ..., S_T$ of a program P (Section 2.3). Each step t is handled by the Main chip and, depending on the instruction, generates rows in other chips: a memory access generates rows in the Memory AIR of its region (and possibly MemAlign), a delegated operation generates a row in the executor chip for its opcode, a precompile call generates as many rows as the precompile needs. For each AIR $\mathcal{A}_i$ of the circuit, let $\text{rows}_i(t)$ denote the finite sequence of rows that step t contributes to $\mathcal{A}_i$; this is a deterministic function of (S_t, ι_t, h_t) —it is the trace generation method of Definition 1 and the chip's witness generator— and the *global trace* of $\mathcal{A}_i$ is the concatenation $\boldsymbol{T}_i^{\text{glob}} := \text{rows}_i(0) \parallel ... \parallel \text{rows}_i(T-1)$, followed for the Memory AIRs by the sorting of Definition 2 and for the table AIRs by the multiplicity counts.

Definition 1 (Segmentation and Segment). Fix trace heights N_i for every AIR. The *segmentation* of the execution is the family of partitions of each global trace $\boldsymbol{T}_i^{\text{glob}}$ into consecutive slices of exactly N_i rows, the last slice padded to N_i rows: $\boldsymbol{T}_i^{\text{glob}} = \boldsymbol{T}_{i,1} \parallel ... \parallel \boldsymbol{T}_{i,m_i}$ with $m_i = \left\lceil |\boldsymbol{T}_i^{\text{glob}}| \, / N_i \right\rceil$ (and $m_i = 0$ if the AIR is unused). A *segment* is a tuple

$$G = (\tau, j, \boldsymbol{T}, \boldsymbol{a}, B^{\text{in}}, B^{\text{out}}),$$

where $\tau \in [M]$ is the AIR type, $j \in [m_\tau]$ the segment index, $(\boldsymbol{T}, \boldsymbol{a}) = \left(\boldsymbol{T}_{\tau,j}, \boldsymbol{a}_{\tau,j}\right)$ the AIR witness of the slice (the AIR values $\boldsymbol{a}$ include j, the last-segment flag $[j = m_\tau]$, and the boundary payloads), and $B^{\text{in}}, B^{\text{out}} \in \mathcal{P}_\tau$ the *boundary payloads* of the chip type, whose payload space $\mathcal{P}_\tau$ is defined in Section 5.3; $\mathcal{P}_\tau = \{\perp\}$ for chips without cross-segment state. The *leaf count* of the execution is $\sum_\tau m_\tau$.

For the Main chip, $m_{\text{Main}} = \lceil T / N_{\text{Main}} \rceil$ and the segment j covers exactly the steps $t \in [(j-1)N, jN)$: the Main's global step is reconstructed inside the AIR as $t = (j-1)N + r$ from the segment index carried in $\boldsymbol{a}$ and the

row r, so the *partition map* from steps to Main segments is the map $t \mapsto \lfloor t/N \rfloor + 1$, enforced by constraint rather than assumed. For every other chip the partition is by row count and does not align with step boundaries; this is precisely why cross-segment consistency cannot be expressed as an equality of whole VM states $S_{\text{end}_j} = S_{\text{start}_{j+1}}$ and is instead decomposed into per-chip boundary payloads plus the globally sorted memory argument, as explained next. The two-level structure is: the Main's payload carries the control state (pc, c), the Memory AIRs' payloads carry the position in the sorted access trace, and the memory *contents* are never carried across a boundary —they are reconstructed by the extractor from the sorted access trace, which spans the whole execution.

Trace splitting eliminates most padding —in fact, it is equivalent to the Jagged PCS approach— but it introduces two problems that must be solved.

The first is *cross-chip consistency*. As commented in Section 4, there are chips that delegate operations to other specialized chips and record the expected result. Proving that the delegation was correct requires that the involved chips agree on the exchanged values. In the standard continuation's solution this agreement is enforced directly by generating a single monolithic proof of the entire trace. In the trace-splitting design the proofs for different chips are generated independently. Binding them into a globally consistent argument requires a shared challenge that spans all chip instances —a check that cannot be performed locally and is instead delegated to the recursive aggregation layer. We explain the solution to this problem in Section 6.

The second is *cross-segment state*. Because each chip's trace is divided independently, a chip whose state evolves over the course of execution cannot restart cold at the beginning of each segment: consecutive segment instances must agree on the machine state at their shared boundary. This is the problem that continuations, in the classical sense, solve.

For chips whose computation is self-contained within each segment — such as base operations— the cross-segment state is empty: consecutive segment instances are independent and carry no shared context. For chips with persistent state, however, consecutive instances must agree on the state at their shared boundary. This shared boundary state is enforced by the *continuation bus*.

5.3. **The Continuation Bus.**

The *continuation bus* carries cross-segment state for each chip that requires it. Each segment instance *receives* the boundary payload it starts from and *sends* the boundary payload it ends with; bus balancing (Section 3.4) ensures the chain is globally consistent: every sent message is received exactly once with matching content, so gaps, duplications, and

reorderings are all ruled out. Two *anchors* —degree-zero terms contributed once per proof rather than per row— open and close each chain: the *genesis* anchor sends the payload the first segment must start from, and the *terminus* anchor receives the payload the last segment must end with. Continuation messages are formed from AIR values (one value per segment, not per row), which the AIR constraints tie to the first and last rows of its trace.

We now give the payload types and message schemas. Throughout, j denotes the segment index, $\text{last} \in \{0, 1\}$ the last-segment flag of the sending segment, and every segment sends exactly one and receives exactly one continuation message.

Main. Payload space $\mathcal{P}_{\text{Main}} = [0, 2^{32}) \times [0, 2^{64})$: the program counter and the value c at the boundary. Bus identifier 1000; messages $(j, \text{last}, \text{pc}, c_0, c_1)$. Segment j receives $(j-1, 0, \text{pc}^{\text{in}}, c^{\text{in}})$ and sends $(j, \text{last}, \text{pc}^{\text{out}}, c^{\text{out}})$, where the AIR constrains pc^{in} to equal the `pc` column at row 0, c^{in} to be the "previous c" used by row 0, pc^{out} to be the next program counter computed by the control-flow rule at row $N-1$, and c^{out} to be the c column at row $N-1$. The genesis anchor sends $(0, 0, \text{pc}_{\text{BOOT}}, 0, 0)$ and the terminus anchor receives $(m_{\text{Main}}, 1, \text{pc}_{\text{END}}, 0, 0)$ —but with the segment-index field of the sent message set to 0 when $\text{last} = 1$, so that the terminus is the unique message with $\text{last} = 1$ and the tail of the last segment cannot be re-consumed as the head of a further one. Registers are not in the payload: they are carried on the Memory Bus by the end-of-segment reload of Section 4.2.1.1, which sends the 31 register values at the last memory step of the segment and is consumed by the first accesses in the next segment.

Memory AIRs. Payload space $\mathcal{P}_{\text{Mem}} = [0, 2^{29}) \times [0, 2^{38}) \times [0, 2^{64})$: the last $(\text{addr}, t_{\text{mem}}, v)$ row of the sorted access trace. Bus identifier 11; messages $(\text{base}, j, (\text{addr}, t_{\text{mem}}, v_0, v_1))$, with the region's base address as first field so that the three regions' chains are disjoint. Segment j receives $(\text{base}, j-1, \text{row}^{\text{in}})$ and sends $(\text{base}, j, \text{row}^{\text{out}})$ if it is not the last; the AIR constrains row^{in} to be the "previous row" of row 0 in Definition 2 and row^{out} to be row $N-1$. The genesis anchor sends $(\text{base}, 0, (\text{base}-1, 0, 0, 0))$ (so that the first row must be an address change), and there is no terminus payload: the last segment simply sends nothing, the region end being enforced by the range check of Definition 2. The anchor is gated by a proof-level boolean that indicates whether the region is used at all.

Other chips. The executor chips and the hash precompiles have $\mathcal{P} = \{\perp\}$ and use no continuation bus. The DMA workers carry the in-flight bulk operation across a boundary (source, destination, remaining count, main step, and for unaligned copies the eight in-flight bytes); their schemas follow the same three-part pattern with their own identifiers.

Definition 2 (Continuation Relation). For a chip type τ with non-trivial payload space, the *continuation relation* $\mathcal{R}^{(\tau)}_{\mathrm{CONT}}$ holds for a sequence of segments $G_1, ..., G_m$ of type τ if

$$B^{\mathrm{in}}_1 = \mathrm{Genesis}_\tau, \quad \forall j < m : B^{\mathrm{out}}_j = B^{\mathrm{in}}_{j+1}, \quad \text{and} \quad B^{\mathrm{out}}_m = \mathrm{Terminus}_\tau \text{ or no terminus is required,}$$

and each G_j satisfies its AIR with $B^{\mathrm{in}}_j, B^{\mathrm{out}}_j$ bound to its first and last rows as described above. For the Main, Genesis $= (\mathrm{pc}_{\mathrm{BOOT}}, 0)$ and Terminus $= (\mathrm{pc}_{\mathrm{END}}, 0)$.

Lemma 3 (Stitching). Let $\{G\}$ be a collection of segments of type τ appearing as leaves of an accepted proof, and suppose the continuation bus of type τ balances (as a component of the global bus balancing of Definition 6). Then the segments can be arranged as a single sequence $G_1, ..., G_m$ satisfying $\mathcal{R}^{(\tau)}_{\mathrm{CONT}}$; in particular there is exactly one segment with index 1 starting from Genesis$_\tau$, exactly one last segment, no cycles, and no two segments with the same index.

The argument is a counting one. Every message on the bus carries the sender's segment index j and is received only by a message with index j in the receiving position; each segment receives exactly one message and sends at most one. The genesis anchor is the unique message with index 0 in the receiving position, so exactly one segment has $j = 1$; by induction, for each j at most one segment receives the message with index $j - 1$, since the sender with index $j - 1$ sends exactly one message, and the multiset equality forces a receiver for every sent message. A cycle would require a message with index j received by a segment claiming index $j + 1$ whose own message is received by a segment of index $j + 2$, and so on, eventually exceeding any index that appears in the finite multiset unless some sender's message is received twice, which the exact multiset equality excludes. The Main's `last` flag prevents the degenerate cycle in which the last segment's output is consumed as the input of a segment of index $m + 1$: the terminus anchor is the only receiver of a message with `last` $= 1$. This lemma is a property of the constraint system and is recorded as a proof obligation in Section 9.5.

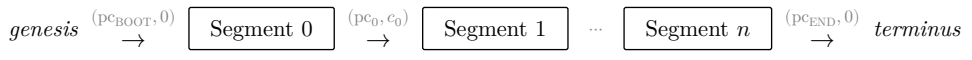


FIGURE 4. Continuation bus for the Main chip. The genesis anchor injects the boot program counter; the terminus anchor receives the final handoff at the designated end program counter.

5.4. **Parallel Proving and Recursive Aggregation.**

Because each segment instance is proved independently, all segments across all chips can be proved in parallel. Each individual proof establishes

only *local consistency*: it certifies that the segment's own constraints are satisfied and records its contribution to the shared bus interactions as a partial sum. Two global checks remain outstanding.

First, the continuation buses must balance globally. Every cross-segment state message sent by a segment instance must be received exactly once by the next, with matching content. No individual segment proof can verify this —it requires aggregating the partial sums from all instances of a chip across all of its segments.

Second, the cross-chip buses must balance globally. The partial sums recording interactions between the Main chip and the Memory, or other specialized chips must cancel to zero when combined. This is the cross-chip consistency problem introduced earlier, and it likewise cannot be enforced locally.

Both checks reduce to a single global field equation: the sum of all partial sums across all instances and all chips must equal zero. The recursive aggregation layer (Section 6) accumulates these partial sums up an aggregation tree and enforces this equation at the root, binding the independent segment proofs into a single, globally valid argument for the full unbounded execution.

6. Recursion

ZisK uses proof recursion to combine the independent per-segment proofs produced by trace splitting into a single, globally consistent argument. This section introduces proof recursion in general, identifies the two specific problems it resolves in ZisK, and details the aggregation structure.

6.1. **Proof Recursion.**

A proving system is called *recursive* when it can verify the validity of another proof as part of its own computation. The key observation is that the verifier V is itself a computation —it reads a proof, the verification key, and the public statement, and performs arithmetic checks— so it can be expressed as polynomial constraints and be proved. A *recursive verifier* is therefore an arithmetic circuit that takes a proof π as witness and a public statement x as input, and checks that $V(\mathrm{vk}, x, \pi)$ accepts for some verification key vk. Wrapping this circuit in a proof yields a *recursive proof*: a succinct argument that attests to the correctness of an underlying proof without re-exposing its content.

The power of this construction comes from its composability. Since a recursive proof is itself a valid proof, it can in turn be fed into another recursive verifier, producing a proof of a proof of a proof. Iterating this idea allows a sequence of computations —or a collection of independent computations— to be folded into a single, constant-size argument regardless of the number of steps or the size of the individual proofs. The result is a

proof composition that scales arbitrarily: adding more computations does not make the final proof larger or more expensive to verify.

Recursive composition is normally used for three purposes. The first is *proof compression*: reducing the size of a proof by recursively re-proving its verifier within the same proving system. The second is *proof aggregation*: combining many proofs $\pi_1, ..., \pi_M$ into a single argument that attests to the validity of all of them simultaneously. The third is *cross-system conversion*: translating a proof generated with one proving system (e.g., a proof with large size but fast to generate) into a proof in another system better suited for a particular context (e.g., a compact proof with fast verification).

ZisK leverages recursion for the previous purposes —trace splitting (Section 5) produces many segment proofs that need to be aggregated; intermediate aggregation proofs are themselves recursively re-proved to shrink their size; and the topmost proof can be converted into a format suitable for the external verifier— but it also relies on it to resolve two open problems left by trace splitting itself.

The first is *cross-chip bus balancing*. Each segment proof contributes a partial sum $s_i \in \mathbb{K}$ to the global bus interaction. Global correctness requires $\sum_i s_i = 0$, but no individual proof can verify this: it requires aggregating the partial sums across all segment instances and all chips.

The second, and more subtle, is *challenge derivation*. The LogUp argument (Section 3) uses random field elements α and γ, derived via the Fiat-Shamir heuristic, to compress bus interactions into a single accumulated value per AIR. For the global balancing equation $\sum_i s_i = 0$ to be meaningful, all partial sums must be computed under the *same* challenges. In a monolithic proof this is automatic: there is one prover transcript and one application of Fiat-Shamir. In the trace-splitting design, each segment is proved independently and would naturally derive its own challenges from its own commitments —producing values that differ across instances and making the partial sums incommensurable.

6.2. **Challenge Derivation.**

The challenge derivation problem arises because Fiat-Shamir ties each challenge to the transcript of a single prover. When segment provers operate independently, their transcripts diverge and so do their challenges.

The solution is to separate challenge derivation from proof generation: all segment provers first commit to their traces, and the challenge is then derived from the full set of commitments and broadcast to all provers as a *public input*. Each segment proof proves the statement "given the challenge α, my witness satisfies my constraints and my partial sum is s_i" —the challenge is *assumed*, not derived internally.

A natural first attempt is plain hashing: Given commitments $\text{com}_1, ..., \text{com}_M$, compute:

$$\alpha = H(\cdots H(H(\text{com}_1, \text{com}_2), \text{com}_3)\cdots, \text{com}_M).$$

This fails for two reasons. It is inherently sequential, which negates the parallelism of trace splitting; and it is order-dependent: changing the processing order changes the challenge, so partial batches cannot be computed independently and merged.

ZisK instead uses a *multiset hash* [Cla+03] —a function $\mathcal{H}$ from multisets over a message space $\mathcal{M}$ to a commutative group $(\mathcal{G}, +)$, satisfying two properties:

- *Multiset Homomorphism*: $\mathcal{H}(S \cup T) = \mathcal{H}(S) + \mathcal{H}(T)$ for any two multisets S, T over $\mathcal{M}$.
- *Multiset Collision-Resistance*: finding distinct multisets $S \neq T$ with $\mathcal{H}(S) = \mathcal{H}(T)$ is computationally infeasible.

The standard construction builds $\mathcal{H}$ from an inner *hash-to-group* function $h : \mathcal{M} \to \mathcal{G}$ as

$$\mathcal{H}(S) = \sum_{x \in S} m_S(x) \cdot h(x),$$

where $m_S(x) \in \mathbb{N}$ is the multiplicity of x in S. As a consequence, segment provers can independently hash and sum their own commitments, and partial accumulators from disjoint subsets can be merged later by addition. The common challenge is then obtained from the global accumulator $\text{acc} = \mathcal{H}(\{\text{com}_1, ..., \text{com}_M\})$ via a deterministic projection $\text{DeriveAlpha} : \mathcal{G} \to \mathbb{K}$, which we instantiate as a Poseidon2 hash of the accumulator vector whose output is interpreted as an element of $\mathbb{K}$:

$$\alpha = \text{DeriveAlpha}(\text{acc}) \in \mathbb{K}.$$

Multiset collision-resistance ensures that the only way for a malicious prover to alter α is to alter the multiset of commitments, which would in turn invalidate at least one segment proof.

ZisK instantiates this scheme with *LtHash* [Zis25], a lattice-based multiset hash. The group is $\mathcal{G}$ is the vector space $\mathbb{Z}_p^n$, and the inner hash $h : \mathbb{F}^k \to \mathbb{Z}_p^n$ is realised by hashing a commitment com to a single field element $c = H(\text{com})$ via Poseidon2 [GKS23], then iteratively expanding c into a long pseudorandom vector $h(\text{com}) \in \mathbb{Z}_p^n$. Multiset collision-resistance for LtHash reduces to the Short Integer Solution (SIS) [Ajt96] lattice problem, which is believed to remain hard even against quantum adversaries. ZisK targets 128-bit security with concrete parameters $n = 372$ and cardinality bound $B = 2^{20}$ (introduced below). A formal statement of the binding property assumed of $\mathcal{H}$, with these parameters, is given in Section 6.2.1.

The LtHash group has finite exponent p: adding p copies of the same element to a multiset leaves H unchanged, since $p \cdot h(x) \equiv 0 \in \mathbb{Z}_p^n$. To rule out such trivial collisions, the scheme is restricted to multisets of cardinality at most $B \ll p$. ZisK enforces this bound by tracking the multiset cardinality alongside the accumulator, via the leaf-count counter κ that the recursion carries and range-checks against B at every aggregation node (introduced in the next section). Throughout, B denotes the bound itself (so the check reads $\kappa < B = 2^{20}$), never an exponent. Equivalently, one may think of LtHash as using an augmented per-element hash $h'(x) = (h(x), 1) \in \mathbb{Z}_p^{n+1}$, so that the last coordinate of the multiset hash tracks the total cardinality automatically.

Correctness of the assumed challenge α must be verified —otherwise a malicious prover could substitute any α to make the partial sums balance artificially. Since $\mathcal{H}$ is homomorphic over multiset union, this verification is embedded directly into the aggregation tree: each node carries a *partial accumulator* over the commitments in its subtree, and a parent node combines the two by addition. At the root, the fully accumulated value is checked against the claimed α, confirming that it was correctly derived from the entire set of commitments.

6.2.1. *Formal Specification of the Challenge Derivation.*

The preceding discussion interleaves the specification with its motivation. We now separate the two and state the challenge derivation as a self-contained primitive with an explicit security requirement, so that the soundness sketch of Section 9 can consume it as a named assumption rather than as prose.

Definition 1 (Multiset Hash and Multiset Binding). Let $\mathcal{M}$ be a message space, $(\mathcal{G}, +)$ a finite abelian group, and $B \in \mathbb{N}$ a *cardinality bound*. A *multiset hash* is a function $\mathcal{H}$ from finite multisets over $\mathcal{M}$ to $\mathcal{G}$ that is *homomorphic*: $\mathcal{H}(S \cup T) = \mathcal{H}(S) + \mathcal{H}(T)$ for all multisets S, T, where $\cup$ is multiset union (multiplicities add). The multiset hash is $(B, \varepsilon_{\mathrm{ms}})$-*binding* if for every probabilistic polynomial-time adversary $\mathcal{A}$,

$$\Pr\big[(S,T) \leftarrow \mathcal{A}(1^\lambda) : S \neq T \wedge |S| \leq B \wedge |T| \leq B \wedge \mathcal{H}(S) = \mathcal{H}(T)\big] \leq \varepsilon_{\mathrm{ms}}(\lambda),$$

where $|S|$ denotes the cardinality of S counted with multiplicity.

Definition 2 (ZisK Challenge Derivation). Let $h : \mathcal{M} \to \mathbb{Z}_p^n$ be the inner hash-to-group function of Section 6.2, and let $\mathcal{H}(S) \coloneqq \sum_{x \in S} m_S(x) \cdot h(x)$ be the induced multiset hash with values in $\mathcal{G} = \mathbb{Z}_p^n$. Let DeriveAlpha : $\mathbb{F}^* \times \mathbb{Z}_p^n \to \mathbb{K}$ be the deterministic projection realised by Poseidon2. Given the public statement $\boldsymbol{p}$ and a multiset S of trace commitments, the *ZisK challenge* is

$$\alpha(\boldsymbol{p}, S) \coloneqq \text{DeriveAlpha}(\boldsymbol{p}, \mathcal{H}(S)) \coloneqq \text{Poseidon2}(\text{Trans}(\boldsymbol{p}) \parallel \mathcal{H}(S)), \quad \gamma \coloneqq H_\gamma(\alpha),$$

where $\text{Trans}(\boldsymbol{p})$ is the canonical transcript encoding of the public statement (the program commitment, the public outputs, and the proof-level selector values of Section 7.1) and $H_\gamma : \mathbb{K} \to \mathbb{K}$ is the fixed hash deriving the compression challenge from α (Section 3.3.1). Including $\boldsymbol{p}$ in the derivation binds the challenge to the statement as well as to the commitments. The concrete parameters are $p = 2^{64} - 2^{32} + 1$, $n = 372$, and $B = 2^{20}$.

The two properties that the recursion relies on are, respectively, the homomorphism (which is what allows $\boldsymbol{c}$ to be accumulated by addition at every internal node) and the following assumption.

Assumption 3 (LtHash Binding). The multiset hash $\mathcal{H}$ of Definition 2 is $(B, \varepsilon_{\text{ms}})$-binding in the sense of Definition 1, with $\varepsilon_{\text{ms}}(\lambda) \leq 2^{-128}$ for the stated parameters, under the hardness of the Short Integer Solution problem for the corresponding lattice parameters [Zis25].

The role of the cardinality bound is the one described earlier: without it, any multiset containing p copies of an element collides with the same multiset with those copies removed, since $\mathcal{G}$ has exponent p. The recursion enforces $|S| \leq B$ by construction —κ counts the leaves under each node and the checks $\kappa < B$ in $\mathcal{R}_{\text{AGG}}$ and $\mathcal{R}_{\text{FINAL}}$ ensure that the multiset whose hash is compared against α at the root has at most B elements— so Assumption 3 applies to it.

Finally, the challenge α is a Fiat-Shamir challenge: it is a deterministic function of the prover's first-round messages (the trace commitments). The standard way to argue about it is to model $\text{DeriveAlpha}(\boldsymbol{p}, \mathcal{H}(\cdot))$ as a random oracle on pairs (statement, multiset of commitments), which is legitimate if $\mathcal{H}$ is binding and DeriveAlpha is modelled as a random oracle: a binding $\mathcal{H}$ means distinct multisets of size at most B map to distinct accumulators except with probability ε_{ms}, after which a random-oracle DeriveAlpha yields independent uniform challenges for distinct (statement, multiset) pairs. This is the form in which Section 9 uses it.

6.3. What ZisK Proves.

Before describing the recursive verifiers, we make explicit what ZisK as a whole proves. Given a fixed program P, ZisK proves that there exists a valid execution of P on some input producing some output. Some elements of

this relation —typically a subset of the program's inputs and a subset of its outputs— can be exposed to the verifier as publics; we collect all such public elements into a single vector $\boldsymbol{p} \in \mathbb{F}^k$, the *public statement*. Everything else (the prover's hints, intermediate memory state, and any non-public output) stays inside the prover.

The execution is split into segments and proved piecewise: given ZisK's circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$, each segment is an instance of one of the AIRs $(\mathcal{A}_i, \mathcal{I}_i) \in \mathcal{C}$ (Definition 1). The segments are stitched together by the global bus-balancing condition (Definition 3) and the global challenge $\alpha \in \mathbb{K}$, derived from the prover's commitments via Fiat-Shamir as described in Section 6.2.

The relations in the rest of this section describe what each recursive verifier checks; they are statements about proof acceptance and bus arithmetic. The formal statement about program semantics —the relation $\mathcal{R}_{\text{exec}}$ whose witness is an execution of P, the ledger of the public statement $\boldsymbol{p}$, and the knowledge-soundness theorem that connects an accepting final proof to a witness of $\mathcal{R}_{\text{exec}}$— is given separately in Section 7, and the reduction from the relations below to $\mathcal{R}_{\text{exec}}$ is sketched in Section 9.

6.4. **Recursive Structure.**

Throughout this section it will be convenient to describe a verifier by the *relation* it attests to. We write a relation as

$$\mathcal{R} = \{(x; w) \mid \varphi(x, w)\},$$

where x is the *public statement* and w is the *witness*; the predicate φ specifies the constraints that must hold. A proof for $\mathcal{R}$ on a given x convinces the verifier that there exists w such that $(x; w) \in \mathcal{R}$.

ZisK assembles its proof as the aggregation tree illustrated in Figure 5. The leaves of the tree are the individual AIR segment instances; the internal nodes are *recursive verifiers* —arithmetic circuits that take their children's proofs as witness, verifies them, and expose a small number of public outputs that propagate upward. Internal nodes have a fixed arity, but the tree is not required to be balanced: it is built opportunistically as proofs complete, so a branch that finishes early can be verified immediately while others are still running. On top of the aggregation tree sits a single *Final Verifier*, which is the only node that performs the global checks —bus balancing across all segments, and consistency of the global challenge α.

Two quantities are accumulated upward at every internal node: the bus partial sum s and the challenge contributions $\boldsymbol{c}$. These accumulations are the mechanism by which the two global checks are reduced, at the Final Verifier, to simple arithmetic on the public outputs that arrive there.

The challenge α is an input to the entire structure, not something derived inside it. Before proving starts, α is derived according to Section 6.2 and

broadcast to every leaf prover. Each leaf proof takes α as a public input and uses it to compute its partial bus sum s. The verifiers then accumulate both s and the challenge contributions $\boldsymbol{c}$ upward, and the Final Verifier receives α as a public input to check that it matches what the tree actually accumulated.

The tree has four kinds of nodes: AIR leaves, leaf verifiers, aggregation verifiers, and the Final Verifier. For the six AIR types whose STARK proofs are largest (Section 8.2), a *compressor* node sits between the AIR leaf and its leaf verifier: it is a recursive verifier for the relation "the AIR proof verifies with this public statement", re-proved at a higher rate so that the leaf verifier circuit stays small. Since a compressor changes neither the statement nor the witness semantics, we fold it into the leaf verifier in the relations below and list it separately only in the topology (Section 9.1).

6.4.1. *AIR Leaves.*

A *leaf* of the aggregation tree corresponds to a particular segment instance of one of the pairs $(\mathcal{A}, \mathcal{I})$ in $\mathcal{C}$. We write $\tau \in [M]$ for the index of that pair —the leaf's *circuit type*— and use the convention that $\mathcal{A}_\tau$ and $\mathcal{I}_\tau$ refer to it.

As public statement, the proof at a leaf takes the program-level publics $\boldsymbol{p}$, the challenge α and the AIR-level summary s. The circuit type τ itself is *not* part of the leaf's public statement: it is determined by which AIR the leaf instantiates and is lifted into the public statement by the leaf verifier above (described next), so that the upper levels of the tree can read it and decide which AIR's verification key to use.

Given $\alpha \in \mathbb{K}$, $\boldsymbol{p} \in \mathbb{F}^k$, $s \in \mathbb{K}$, a proof for a leaf of circuit type τ attests that there exists $\boldsymbol{T} \in \mathbb{F}^{N \times W}$, $\boldsymbol{a} \in \mathbb{F}^V$ such that:

- The witness $(\boldsymbol{T}, \boldsymbol{a})$ satisfies the AIR $\mathcal{A}_\tau$, and is consistent with the program-level publics $\boldsymbol{p}$ —i.e., the cells of the trace and AIR values that the AIR designates as bindings to the program's publics match $\boldsymbol{p}$.
- The partial bus sum s is correctly computed from the AIR's interactions $\mathcal{I}_\tau$ and $(\boldsymbol{T}, \boldsymbol{a})$ under the challenge α.

In the language of relations:

$$\mathcal{R}^{(\tau)}_{\text{AIR}} = \left\{ (\boldsymbol{p}, \alpha, s); (\boldsymbol{T}, \boldsymbol{a}) \;\middle|\; \begin{array}{l} (\boldsymbol{T}, \boldsymbol{a}) \text{ satisfies } \mathcal{A}_\tau \text{ and is consistent with } \boldsymbol{p} \\ \wedge \qquad\qquad s = \text{BusCont}_\alpha(\mathcal{I}_\tau, \boldsymbol{T}, \boldsymbol{a}) \end{array} \right\}$$

where $\text{BusCont}_\alpha(\mathcal{I}_\tau, \boldsymbol{T}, \boldsymbol{a})$ is the AIR contribution to the bus, computed from the set of interactions $\mathcal{I}_\tau$ and the witness $(\boldsymbol{T}, \boldsymbol{a})$ under α (as defined in Section 3.3).

6.4.2. *Leaf Verifier.*

A *leaf verifier* sits directly above an AIR leaf. It takes the AIR proof π as witness, runs the AIR verifier on it, and re-exposes the AIR proof's public statement, augmented with three new publics that the recursive layer above will rely on: the circuit type $\tau \in [M]$, a counter $\kappa = 1$, and the challenge contribution $\boldsymbol{c} \in \mathbb{Z}_p^n$. There is one leaf verifier circuit per pair $(\mathcal{A}_i, \mathcal{I}_i) \in \mathcal{C}$: a leaf verifier for the AIR of circuit type τ runs the verifier $V_{\text{AIR}}^{(\tau)}$, which is hard-wired to the verification key of $\mathcal{A}_\tau$, and assigns the constant τ —fixed by which AIR the leaf verifier is for, not chosen by the prover. The counter κ will, as it propagates up through aggregation, track how many leaf proofs have been combined under each subtree, enforcing the cardinality bound required by the multiset hash of Section 6.2. The challenge contribution $\boldsymbol{c}$ is the leaf's contribution to the global accumulator: the leaf verifier computes it itself by applying the inner hash-to-group h of Section 6.2.

The challenge contribution is computed from the *trace commitment* carried inside the AIR proof. We write $\text{com}(\pi)$ for the commitment component of an AIR proof π: the backend of Section 8 produces, as the first message of every proof, a Merkle root binding the committed trace columns of $(\boldsymbol{T}, \boldsymbol{a})$, and $\text{com}(\pi)$ denotes exactly that root together with the identifier of the AIR the proof is for (Section 8.3 gives the precise contents). The function $\text{com}(\cdot)$ is a fixed, public parsing of the proof string; it involves no secret and no choice by the prover beyond the proof itself, so it is a legitimate helper function inside a relation whose witness is π.

The leaf verifier additionally carries an extra public vk —a verification key— which it does not use itself. The reason is circular: as we will see, an aggregation verifier may need to verify a proof of *itself* as one of its children, and a circuit cannot hard-code its own verification key without inducing a circular dependency at setup time. ZisK breaks this loop by treating that key as a public that is *propagated* through every level of the tree, with the Final Verifier closing the loop by fixing vk to a hard-coded constant. The leaf verifier is the bottom of this chain: it has no aggregator below it to verify, so vk is just declared so that the aggregation verifier above can read it.

For brevity we bundle the four recursion-tracking publics into a single tuple

$$\sigma = (s, \tau, \kappa, \boldsymbol{c}),$$

which we call the *recursive statement*. Every recursive verifier (leaf, aggregation, and Final) carries its own σ in its public statement, and aggregation verifiers read their children's σ's as part of the witness. We also adopt the shorthand $V_X(x, \pi)$ for $V(\text{vk}_X, x, \pi)$, where vk_X is the verification key of relation $\mathcal{R}_X$. A proof for the leaf verifier of circuit type τ attests that:

$$\mathcal{R}^{(\tau)}_{\text{LEAF}} = \left\{ (\boldsymbol{p}, \alpha, \sigma, \text{vk}); \pi \;\middle|\; \begin{array}{ll} V^{(\tau)}_{\text{AIR}}((\boldsymbol{p}, \alpha, s), \pi) = \text{accept} \\ \wedge \qquad\qquad \boldsymbol{c} = h(\text{com}(\pi)) \\ \wedge \qquad\qquad \kappa = 1 \end{array} \right\}$$

where $V^{(\tau)}_{\text{AIR}}$ is the verifier induced by $\mathcal{R}^{(\tau)}_{\text{AIR}}$. The bus contribution s is checked by $V^{(\tau)}_{\text{AIR}}$, so accepting the AIR proof binds it to the value committed in π, which the leaf verifier then forwards unchanged into its public statement. The challenge contribution $\boldsymbol{c}$, on the other hand, is computed by the leaf verifier itself, since h only depends on the trace commitment $\text{com}(\pi)$, which is a field of π that the leaf verifier parses directly. The remaining components are constants associated with this leaf verifier: τ is fixed by the choice of $V^{(\tau)}_{\text{AIR}}$, and $\kappa = 1$ marks "this subtree contains one leaf". Finally, vk is carried as a public but unused at this level. Every symbol in the displayed relation is thus bound: $\boldsymbol{p}, \alpha, \sigma, \text{vk}$ by the public statement, π by the witness, τ and the constant 1 by the choice of leaf-verifier circuit, and $V^{(\tau)}_{\text{AIR}}, h, \text{com}$ are fixed public functions.

6.4.3. *Aggregation Verifier.*

An *aggregation verifier* has a fixed arity k —$k = 3$ in ZisK v0.16.1— and takes the proofs $\pi_1, ..., \pi_k$ produced by its children as witness, runs the recursive verifier on each of them, and accumulates the bus partial sums s_i and the challenge contributions $\boldsymbol{c}_i$ upward. When fewer than k proofs are available, the remaining child slots are *null*: a null child is marked by the reserved circuit type $\tau = 0$, is not verified, and contributes $s = 0$, $\boldsymbol{c} = \boldsymbol{0}$, $\kappa = 0$. The contribution $\boldsymbol{c} = \sum_i \boldsymbol{c}_i$ is exactly the multiset-homomorphism step of the multiset hash $\mathcal{H}$: under the leaf-element interpretation given above, summing the children's $\boldsymbol{c}$'s computes $\mathcal{H}$ on the union of the subtrees' multisets. The accumulated $(s, \boldsymbol{c})$ propagate up the tree until the root.

Each non-null child of an aggregation verifier is itself either a leaf verifier or another aggregation verifier. The aggregation verifier therefore reads each child's circuit type τ and selects the appropriate verifier accordingly: the per-AIR verifier $V^{(\tau)}_{\text{LEAF}}$ when $\tau \in [M]$, the aggregation verifier V_{AGG} when $\tau = M + 1$, and no verifier at all when $\tau = 0$. We denote this selection by $V^{(\tau)}_{\text{REC}}$, with the convention that $V^{(0)}_{\text{REC}}$ accepts exactly the null statement $\sigma = (0, 0, 0, \boldsymbol{0})$. The aggregation verifier exposes its own circuit type $\tau = M + 1$ in its public statement to mark the resulting proof as "aggregated", so the level above dispatches to V_{AGG} when verifying it. The circuit type is thus *dispatch information determined by the child's provenance*: a leaf verifier of type τ hard-codes τ into its public statement, an aggregation verifier hard-codes $M + 1$, and neither is a free choice of the prover. The parent enforces the consequence: whatever τ a child declares, the parent runs the verifier with the verification key *for that* τ, so a proof produced for one circuit type cannot be passed off as another.

Dispatching also requires the aggregation verifier to know the verification key of whichever inner verifier it ends up running. The keys $\mathrm{vk}_{\mathrm{LEAF}}^{(1)}, ..., \mathrm{vk}_{\mathrm{LEAF}}^{(M)}$ of the per-circuit-type leaf verifiers are fixed at setup time and are hard-coded into the aggregation circuit. The aggregation verifier's *own* key is exactly the public vk introduced above: this is the value that lets the aggregator verify aggregation children. The aggregator additionally binds each of its children to declare the same value of vk as itself, so the same value propagates uniformly through the entire tree.

A proof for an aggregation verifier attests that:

- Each child proof π_i verifies under the verifier selected by its circuit type $\tau_i \in \{0\} \cup [M+1]$ and under its public statement.
- The partial bus sum s is the sum of the children's partial sums, $s = \sum_i s_i$.
- The challenge contribution $\boldsymbol{c}$ is the sum of the children's contributions, $\boldsymbol{c} = \sum_i \boldsymbol{c}_i$.
- The leaf-count counter κ is the sum of the children's counters, $\kappa = \sum_i \kappa_i$, and both every child counter and every running partial sum of the addition are range-checked against $B = 2^{20}$ (Section 6.2); the checks are performed in the circuit by a 20-bit decomposition. Checking the sum after every addition is what makes the bound hold at the root, where κ equals the total number of leaves in the tree.
- Both children declare the same vk as the parent.
- The exposed circuit type is the aggregation marker $\tau = M + 1$.

Formally:

$$\mathcal{R}_{\mathrm{AGG}} = \left\{ (\boldsymbol{p}, \alpha, \sigma, \mathrm{vk}); ((\sigma_i, \pi_i))_{i \in [k]} \;\middle|\; \begin{array}{ll} & \forall i \in [k] : V_{\mathrm{REC}}^{(\tau_i)}((\boldsymbol{p}, \alpha, \sigma_i, \mathrm{vk}), \pi_i) = \mathrm{accept} \\ \wedge & \tau = M + 1 \\ \wedge & \kappa = \sum_i \kappa_i \wedge \forall i : \kappa_i < B \wedge \sum_{i' \le i} \kappa_{i'} < B \\ \wedge & s = \sum_i s_i \\ \wedge & \boldsymbol{c} = \sum_i \boldsymbol{c}_i \end{array} \right\}$$

where $V_{\mathrm{REC}}^{(\tau)} = V_{\mathrm{LEAF}}^{(\tau)}$ if $\tau \in [M]$, $V_{\mathrm{REC}}^{(\tau)} = V_{\mathrm{AGG}}$ if $\tau = M + 1$, and $V_{\mathrm{REC}}^{(0)}$ accepts only the null statement. Each $V_{\mathrm{REC}}^{(\tau)}$ is invoked with vk in the child's public statement, so by accepting that proof the aggregator implicitly checks that the child declared the same vk as itself. The verification keys used by $V_{\mathrm{LEAF}}^{(1)}, ..., V_{\mathrm{LEAF}}^{(M)}$ are constants of the aggregation circuit; the key used by V_{AGG} is the public vk (Section 8.4).

Remark (Worked example of dispatch). *Consider a Main leaf and a Keccakf leaf feeding one aggregation node, with the third slot null. The Main leaf verifier is the circuit of type $\tau = 1$; it verifies the Main AIR proof with the hard-coded key $\mathrm{vk}_{\mathrm{AIR}}^{(1)}$, hashes the proof's trace commitment into $\boldsymbol{c}_1$, and outputs $\sigma_1 = (s_1, 1, 1, \boldsymbol{c}_1)$. The Keccakf leaf verifier (type $\tau = 15$, say) does the same with $\mathrm{vk}_{\mathrm{AIR}}^{(15)}$ and outputs $\sigma_2 = (s_2, 15, 1, \boldsymbol{c}_2)$. The aggregation node reads $\tau_1 = 1$ and $\tau_2 = 15$ from the children's statements, verifies π_1 with the constant $\mathrm{vk}_{\mathrm{LEAF}}^{(1)}$ and π_2 with the constant $\mathrm{vk}_{\mathrm{LEAF}}^{(15)}$, checks that both children carry the same* vk *as its own public, treats the third child as null, and outputs $\sigma = (s_1 + s_2, M + 1, 2, \boldsymbol{c}_1 + \boldsymbol{c}_2)$. A prover cannot present the Keccakf proof as a Main proof: the leaf verifier that produced σ_2 hard-codes $\tau = 15$, and if the prover instead ran the type-1 leaf verifier on the Keccakf proof, $V_{\mathrm{AIR}}^{(1)}$ would reject it, since the two AIRs have different verification keys.*

6.4.4. *Final Verifier.*

The *Final Verifier* sits above the aggregation tree. It takes the topmost aggregation proof produced by the tree as its single child, runs the aggregation verifier V_{AGG} on it, and additionally enforces the two global checks. Because the topmost node of the aggregation tree is always an aggregation verifier, the Final expects its child to advertise $\tau = M + 1$; this saves the Final from having to dispatch over circuit types as the internal aggregators do. The Final also closes the recursion loop on vk: it carries a *fixed parameter* vk^* —a hard-coded constant, the actual aggregation verifier's verification key fixed at setup time— and binds the child's declared vk to that constant. Since vk^* is neither part of the public statement nor of the witness, we write it as a parameter of the relation, $\mathcal{R}_{\mathrm{FINAL}[\mathrm{vk}^*]}$; the external verifier knows vk^* as part of the ZisK verification context (Section 8.4). After this point s, $\boldsymbol{c}$, τ, and κ have served their purpose and are no longer carried forward: they are absorbed into the global checks below, and only the program-level public statement $\boldsymbol{p}$ remains visible to an external verifier.

A proof for the Final Verifier attests that:

- The child's circuit type is $\tau = M + 1$, i.e., the child is an aggregation proof.
- The child aggregation proof π verifies under the public statement $(\boldsymbol{p}, \alpha, \sigma, \mathrm{vk}^*)$; since the child's vk is a public of the child, running V_{AGG} with vk^* in that position is what binds the child's declared key to the Final's hard-coded constant.
- The leaf-count counter κ is less than B, confirming that the multiset of commitments aggregated under the child satisfies the cardinality bound required by $\mathcal{H}$ (see Section 6.2).
- Bus balancing holds globally: $s = 0$.

- The challenge α was honestly derived from the public statement and the full set of commitments: $\alpha = \text{DeriveAlpha}(\boldsymbol{p}, \boldsymbol{c})$, the deterministic projection of Definition 2 applied to the accumulated $\boldsymbol{c}$.

Formally, the Final Verifier attests to the relation:

$$\mathcal{R}_{\text{FINAL}[\text{vk}^*]} = \left\{ \boldsymbol{p}; (\alpha, \sigma, \pi) \;\middle|\; \begin{array}{c} V_{\text{AGG}}((\boldsymbol{p}, \alpha, \sigma, \text{vk}^*), \pi) = \text{accept} \\ \wedge\, \kappa < B \\ \wedge\, s = 0 \\ \wedge\, \alpha = \text{DeriveAlpha}(\boldsymbol{p}, \boldsymbol{c}) \end{array} \right\}$$

Remark. *In the degenerate case of an execution that fits in a single segment, the aggregation tree is empty: there is nothing to aggregate. The Final Verifier then takes the leaf verifier's proof directly as its child, dispatching to $V_{\text{LEAF}}^{(\tau)}$ in exactly the same way as an aggregation verifier would. The relation $\mathcal{R}_{\text{FINAL}}$ above generalises to this case by replacing V_{AGG} with $V_{\text{LEAF}}^{(\tau)}$ and accepting any $\tau \in [M]$; everything else stays the same.*

The two equality checks have direct global meaning. The first, $s = 0$, is the global bus-balancing condition from Definition 3 —every message sent on every bus across every segment has been received exactly once with matching values. The second, $\alpha = \text{DeriveAlpha}(\boldsymbol{p}, \boldsymbol{c})$, is *challenge correctness*: by the multiset homomorphism of $\mathcal{H}$, the value of $\boldsymbol{c}$ at the top of the tree equals $\sum_i h(\text{com}(\pi_i)) = \mathcal{H}(S)$ over the multiset S of segment trace commitments, so this check confirms that α was honestly derived from the statement and the full set of commitments and not chosen adversarially. Both checks are *commit-then-justify* patterns: every leaf uses α before the multiset it is derived from has been assembled, and every aggregator uses vk before the Final fixes it. Their soundness rests on the two root checks being enforced on the very values that propagated up the tree, which is argued in Section 9.

Both checks reduce to simple arithmetic on values propagated up the tree as publics, with no need to inspect any individual AIR trace. This layered structure achieves a clean separation of responsibilities: AIR leaves prove only local constraint satisfaction; leaf verifiers convert proofs to a recursion-friendly format; aggregation verifiers accumulate both running totals; and the Final Verifier enforces global bus balancing and challenge integrity. Because every node can proceed as soon as its children complete, the entire proving pipeline is fully parallelizable —no single AIR needs to handle the entire computation, eliminating the bottleneck of a monolithic circuit.

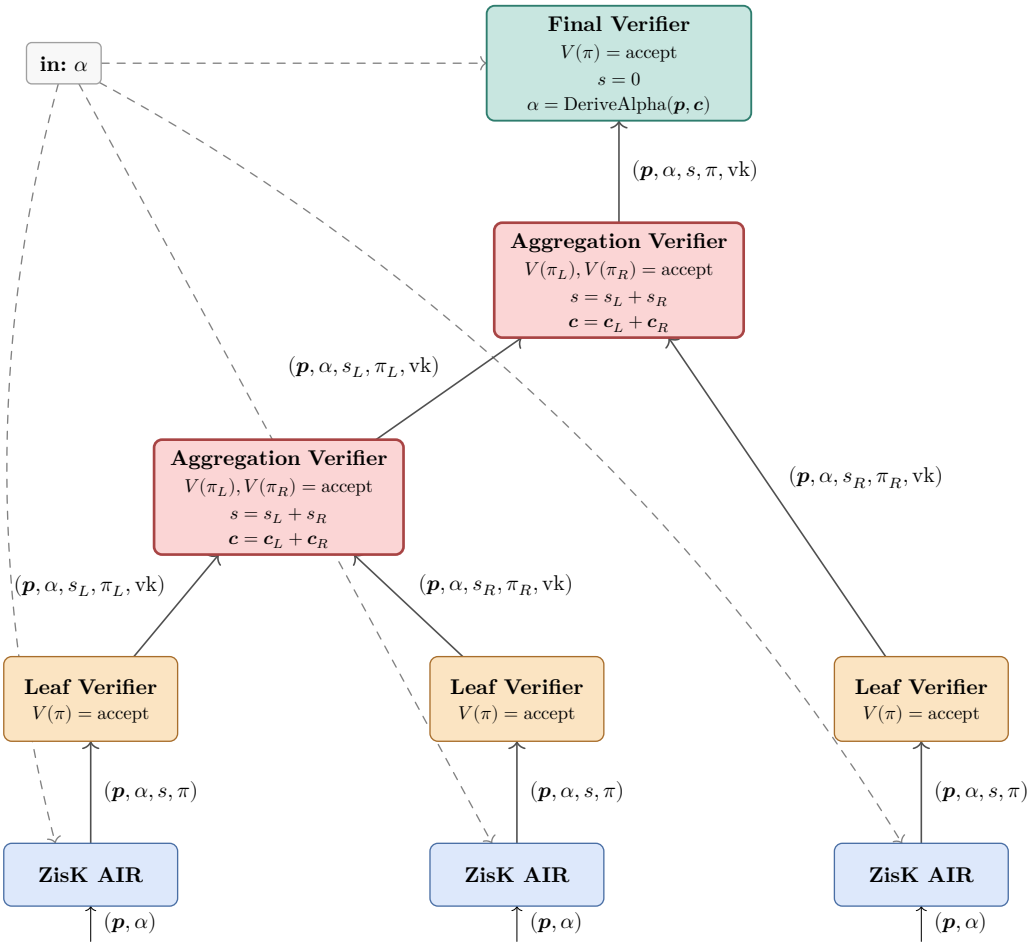


FIGURE 5. Example aggregation tree (drawn with two children per node for readability; ZisK's aggregation verifier has arity 3, unused slots being null children). The tree is unbalanced. Dashed arrows carry the program-level challenge α down to the leaves. Solid arrows carry inputs and outputs to each level: each node's outgoing label is its output, and the incoming labels at any node are its inputs.

7. The ZisK Statement

The relations of Section 6 say what each recursive verifier checks: that certain proofs verify and that certain sums vanish. They do not, on their face, say anything about programs. This section states what ZisK proves *about program execution*: a typed public statement, a semantic execution relation whose witness is an execution in the sense of Section 2.3, and the knowledge-soundness property that connects an accepting final proof to such a witness. The sketch of why the relations of Section 6 imply this property is the subject of Section 9.

7.1. **Public Statement Ledger.**

The external verifier receives the Final Verifier's proof together with a vector $\boldsymbol{p} \in \mathbb{F}^{68}$ and a fixed *verification context*. Table 5 says what every entry is and which role it plays.

Entry	Width	Role	How it is bound
root_{ROM}	4	program binding	Merkle root of the cached ROM columns encoding P (Table 8). Consumed by the ROM AIR's verifier as the root of its custom commitment, so every instruction fetched on the ROM Bus is a row of the table committed under this root.
$\text{out}_0, ..., \text{out}_{31}$ as 64 limbs	64	public outputs	Received on the Operation Bus as the 32 messages $(\texttt{pubout}, i, \text{out}_i, \text{out}_i, 0, 0, 0)$ by a degree-zero global term (Section 4.4.1); balancing forces the Main to have executed exactly one `pubout` per index with matching value.
vk_{FINAL}, vk^*	context	verification keys	Held by the external verifier; fix the whole key chain of Table 8 and thereby the circuit $\mathcal{C}$ and the backend parameters.
α	3	recursive only	The global challenge. It is a public of every internal node but is *consumed* by $\mathcal{R}_{\text{FINAL}}$ ($\alpha = \text{DeriveAlpha}(\boldsymbol{p}, \boldsymbol{c})$) and does not appear in the external statement.
$\sigma = (s, \tau, \kappa, \boldsymbol{c})$	$1 + 1 + 1 + 372$	recursive only	Recursive statement; consumed by the Final's global checks and not exposed externally.
vk (propagated)	4	recursive only	Bound to vk^* by the Final; not exposed externally.
proof-level selectors	9	witness-only	Booleans indicating which optional AIRs (input data, ROM data, DMA variants) are instantiated; constrained boolean, included in the challenge transcript, and used only to gate the genesis anchors of unused regions. They carry no semantic information about the execution.

TABLE 5. Ledger of the public statement. The externally meaningful part is $x := (\text{root}_{\text{ROM}}, \text{out})$, and $\boldsymbol{p}$ is its field encoding; the remaining rows exist only inside the recursion. Neither the input stream nor the hints appear anywhere: they are existentially quantified in Definition 1.

We write $x = (\mathrm{root}_{\mathrm{ROM}}, \mathrm{out}) \in \mathbb{F}^4 \times \left[0, 2^{64}\right)^{32}$ for the typed final statement and identify it with $\boldsymbol{p}$ through the encoding of Table 5.

7.2. **The Execution Relation.**

Let $\mathrm{Commit}_{\mathrm{ROM}(P)}$ denote the deterministic procedure that lays out a program P as the cached columns of the ROM AIR (one row per instruction with the fields of Section 2.2.2, padded with zero rows to 2^{22}) and returns their Merkle root under the backend's commitment (Section 8.3). We use the objects of Section 2.3: guest states $S_t = (\mathrm{pc}_t, c_t, \mathrm{mem}_t)$, instructions ι_t, hint sequences h_t, the step predicate Step_P, the initial memory $\mathrm{mem}_0(P, \mathrm{in})$, and the outputs $\mathrm{out}_i(S_0, ..., S_T)$.

Definition 1 (ZisK Execution Relation). The *execution relation* $\mathcal{R}_{\mathrm{exec}}$ consists of the pairs $(x; w_{\mathrm{exec}})$ with $x = (\mathrm{root}_{\mathrm{ROM}}, \mathrm{out})$ and

$$w_{\mathrm{exec}} = \left(P, T, \mathrm{in}, (S_t)_{t=0}^{T}, (\iota_t)_{t=0}^{T-1}, (h_t)_{t=0}^{T-1}\right)$$

such that

1. (*ProgramBind*) $\mathrm{root}_{\mathrm{ROM}} = \mathrm{Commit}_{\mathrm{ROM}(P)}$;
2. (*Init*) $S_0 = (\mathrm{pc}_{\mathrm{BOOT}}, 0, \mathrm{mem}_0(P, \mathrm{in}))$, with in a word-aligned byte string of length less than 2^{30};
3. (*Fetch and Step*) for all $t < T$: $\iota_t = P[\mathrm{pc}_t]$ and $\mathrm{Step}_P(S_t, \iota_t, h_t, S_{t+1})$ (Definition 1);
4. (*Term*) $\mathrm{pc}_T = \mathrm{pc}_{\mathrm{END}}$ and $c_T = 0$, and $T < 2^{36}$;
5. (*Bind*) $\mathrm{out}_i = \mathrm{out}_i(S_0, ..., S_T)$ for all $i \in [0, 32)$, i.e. the i-th public output declared by the execution equals the i-th word of the statement.

In words: an accepting final proof for x is meant to convince the verifier that *there exist* an input and a hint sequence under which the program committed to by $\mathrm{root}_{\mathrm{ROM}}$ runs successfully from the boot state and declares exactly the outputs out. The input and the hints are witnesses, not statement: ZisK proves nothing about them beyond what the program itself checks and exposes through its outputs. The relation is about the committed ZisK program P; its connection to a RISC-V binary is the separate assumption of Section 9.3.

The two binding predicates of the reviewers' template are instantiated as follows. $\mathrm{Bind}_{\mathrm{in}}$ is vacuous —no public input is bound— except for the program, which is bound through $\mathrm{root}_{\mathrm{ROM}}$. $\mathrm{Bind}_{\mathrm{out}}$ is the equality of the statement's words with the values carried by the 32 `pubout` instructions of the execution; since those instructions read the output-data region of mem_T word by word, $\mathrm{Bind}_{\mathrm{out}}$ equivalently says that $\mathrm{out}_i = \mathrm{mem}_T[$ `0xA001_0000` $+8i]_8$ for the memory at the time the boot code reads it.

7.3. **Execution Extractability.**

Let $\Pi_{\text{fin}} = (\text{Prove}_{\text{fin}}, V_{\text{fin}})$ be the final non-interactive argument exposed to the external verifier: $V_{\text{fin}(x,\pi)}$ runs the Final Verifier's STARK verifier $V(\text{vk}_{\text{FINAL}}, \boldsymbol{p}, \pi)$ with $\boldsymbol{p}$ the encoding of x. We use the standard knowledge-soundness notion with a straight-line extractor, following the reference formulation of the EF sprint documents [Eth26a].

Definition 2 (Execution Extractability). Π_{fin} is *execution-extractable* for $\mathcal{R}_{\text{exec}}$ with error ε_{fin} if there is a probabilistic polynomial-time extractor $\mathcal{E}_{\text{fin}}$ such that for every probabilistic polynomial-time adversary $\mathcal{A}$,

$$\Pr\left[(x,\pi) \leftarrow \mathcal{A}(1^\lambda) : V_{\text{fin}(x,\pi)} = \text{accept} \wedge \left(x; \mathcal{E}_{\text{fin}(x,\pi)}\right) \notin \mathcal{R}_{\text{exec}}\right] \leq \varepsilon_{\text{fin}(\lambda)}.$$

The extractor is given (x, π) and the same oracle access as the verifier; its running time is that of $\mathcal{A}$ plus a polynomial in λ and in the size of the extracted execution.

Remark (Extraction model). *ZisK's proofs are hash-based, and all challenges (the per-proof STARK challenges and the global challenge α) are derived by Fiat-Shamir from Poseidon2. The natural model is therefore the random-oracle model for Poseidon2, in which STARK/FRI arguments admit straight-line extractors. Composing such extractors across the recursion requires each inner argument to remain knowledge-sound relative to the same oracle, which is known not to hold for succinct arguments in general [BCG24]. Like the reference vanilla analysis, we idealise over this obstruction: the soundness sketch of Section 9 assumes straight-line extraction composes across the tree, and the resulting bounds should be read as statements about the reduction structure rather than as concrete security levels for the recursion. The concrete* 128*-bit figure of Section 8.2 is the soundness of each individual STARK, which is what* `soundcalc` *measures.*

Theorem 3 (What ZisK Proves (target statement)). Under the assumptions listed in Section 9.5 —knowledge soundness of the backend STARK for every circuit in Table 9, binding of the trace commitments, LtHash binding (Assumption 3), and the random-oracle modelling of Poseidon2— the ZisK final argument Π_{fin} is execution-extractable for $\mathcal{R}_{\text{exec}}$, with

$$\varepsilon_{\text{fin}} \leq \sum_{\text{nodes } v} \varepsilon_{\text{be}(v)} + \varepsilon_{\text{ms}} + \varepsilon_{\text{FS}} + \varepsilon_{\text{LogUp}} + \varepsilon_{\text{bind}},$$

where the sum ranges over the nodes of the aggregation tree (each contributing the knowledge-soundness error of its STARK), ε_{ms} is the LtHash binding error, ε_{FS} the probability that a random-oracle challenge collides or is guessed, $\varepsilon_{\text{LogUp}}$ the bus soundness error of Definition 6 at a random α, and $\varepsilon_{\text{bind}}$ the Merkle binding error over all commitments opened by the extractor.

The theorem is the W2 *target*: its statement is precise, its proof is the W3 deliverable. Section 9 gives the lemma chain and the ledger of every term used above, indicating for each whether it is discharged in this document, inherited from a standard result, or left as an explicit W3 obligation.

7.4. **Definition Ledger.**

The following table lists every soundness-flavoured term used in Theorem 3 and in Section 9, and where its meaning is fixed.

Term	**Status**	**Where**
execution relation $\mathcal{R}_{\text{exec}}$	defined here	Definition 1
step predicate, fetch, initial and terminal states	defined here	Definition 1 and Section 2.3
execution extractability	defined here	Definition 2
backend knowledge soundness ε_{be}	assumed (backend)	Section 8.1, values from `soundcalc` [Eth26b]
trace-commitment binding	assumed (backend)	Section 8.3: Merkle binding under collision-resistant Poseidon2
LtHash multiset binding ε_{ms}	defined here, assumed	Definition 1, Assumption 3
Fiat-Shamir / transcript binding ε_{FS}	assumed (random oracle)	Definition 2 and the remark on the extraction model
bus balancing (semantic)	defined here	Definition 3
global bus relation $\mathcal{R}^{\alpha}_{\text{BUS}}$ and LogUp soundness $\varepsilon_{\text{LogUp}}$	defined here, proof obligation	Definition 6 and Section 9.5
per-bus relations (operation, ROM, memory, table)	defined here	Section 4.4
memory argument, register and MemAlign refinements	defined here, proof obligation	Definition 2, Lemma 3, Lemma 4
segment, continuation relation, stitching	defined here, proof obligation	Definition 1, Definition 2, Lemma 3
recursive relations $\mathcal{R}_{\text{AIR}}$, $\mathcal{R}_{\text{LEAF}}$, $\mathcal{R}_{\text{AGG}}$, $\mathcal{R}_{\text{FINAL}}$	defined here	Section 6
transpiler correctness	assumed (trusted computing base)	Section 9.3

TABLE 6. Definition ledger for the terms used in the soundness statement.

8. Proving Backend

The arithmetization framework of Section 3 is backend-independent by design, and the preceding sections have used the backend only through the abstract interface $V(\mathrm{vk}, x, \pi)$. For the soundness analysis, however, the backend cannot remain abstract: the binding of trace commitments, the soundness error of each proof, and the parameters that an external reviewer needs in order to evaluate the system (for instance, in the EF `soundcalc` tool [Eth26b]) all depend on it. This section fixes the backend that ZisK v0.16.1 actually uses and states its parameters. Nothing in the previous sections depends on these choices, so the reader interested only in the architecture may skip ahead.

8.1. **The STARK Backend.**

Every relation in this paper —the AIR leaves $\mathcal{R}_{\mathrm{AIR}}^{(\tau)}$ as well as the recursive relations $\mathcal{R}_{\mathrm{LEAF}}^{(\tau)}$, $\mathcal{R}_{\mathrm{AGG}}$ and $\mathcal{R}_{\mathrm{FINAL}}$— is proved with the same proving system: a univariate STARK following the DEEP-ALI paradigm [Ben+18b, Sta23], with FRI [Ben+18a] as low-degree test and Merkle trees as vector commitments, made non-interactive with the BCS transform [BCS16]. Concretely, for an AIR of width W, depth D and trace height N over the domain $L_0 = \langle \omega \rangle \subset \mathbb{F}^\times$ of size N:

1. **Trace commitment.** The witness columns of $\boldsymbol{T}$ are interpolated to polynomials of degree $< N$, evaluated on a coset of a larger domain L of size $\frac{N}{\rho}$ (the *rate* $\rho = 2^{-k}$ is the inverse of the blowup factor), and committed by a Merkle tree whose leaves are the rows of the evaluation matrix. The tree hash is Poseidon2 over $\mathbb{F}$ with a 256 -bit digest (four field elements) [GKS23]. The root of this tree is the *stage-1 commitment*; it is the object $\mathrm{com}(\pi)$ of Section 6 (see Section 8.3). Fixed columns are committed once at setup and their root is part of the verification key; cached columns (Section 3) are committed once per program and their root is bound as described in Section 8.4.
2. **Auxiliary commitment.** After the verifier's challenges $\alpha, \gamma \in \mathbb{K}$ are available, the prover commits (in a second Merkle tree) to the LogUp columns of Section 3.3.1: for each interaction the column of inverses $1/(\alpha + \hat{e}(r))$ and the partial-sum column S. In ZisK, α and γ are not sampled from the transcript of the individual proof but are the global challenge of Section 6.2, supplied as a public input; this is a deviation from the plain STARK and is analysed in Section 9.2.
3. **Constraint aggregation (ALI).** The verifier samples a challenge and the prover commits to the quotient polynomial of the random linear combination of all constraint polynomials, divided by the

vanishing polynomial of L_0. The maximum constraint degree d (three for the ZisK AIRs, up to eight for the recursion circuits) determines the number of quotient chunks.

4. **DEEP query and FRI.** The verifier samples an out-of-domain point $z \in \mathbb{K}$; the prover sends the evaluations of all committed polynomials at z (and at the shifted points $\omega^k z$ for $k \leq D$), and the two sides run FRI on the random linear combination of the DEEP quotients to certify that the committed evaluations are close to low-degree polynomials. FRI uses folding factors as listed in Table 7, an early-stop degree, a number of query rounds q, and proof-of-work *grinding* of g bits before the query phase.

The extension field $\mathbb{K} = \mathbb{F}_{p^3}$ is used for every verifier challenge and every out-of-domain evaluation, so that the soundness error of each random-challenge step is of order $\frac{N}{|}\mathbb{K}| \approx N \cdot 2^{-192}$.

Circuit family	**$\log_2 N$**	**rate ρ**	**queries q**	**grinding g**	**folding**
ZisK AIRs (Main, Mem, Rom, Binary, …)	20–22	1/2	229–232	16 (20 Rom, 23 Keccakf)	$[8, ..., 8]$ or $[8, ..., 8, 4]$
Poseidon2 AIR	17	1/4	114	16	$[8, 8, 8, 8, 4]$
Compressors	18–20	1/4	110	20	$[8, 8, 8, 8, 8]$ or $[8, 8, 8, 8, 8, 4]$
Aggregation verifier (`Recursive2`)	17	1/8	73	20	$[8, 8, 8, 8, 8]$
Final verifier (`Final`)	16	1/32	43	22	$[16, 16, 16, 16]$
Final, compressed (`Final_Compressed`)	15	1/16	54	22	$[8, 8, 8]$

TABLE 7. Backend parameters of ZisK v0.16.1 by circuit family, as registered in `soundcalc` [Eth26b]. All circuits use the Goldilocks field with cubic extension, Poseidon2 Merkle trees with 256-bit digests, DEEP-ALI with powers-based batching, and univariate LogUp for every bus. Per-circuit values are listed in Table 9.

8.2. **Security Level and Soundness Regime.**

With the parameters of Table 7, `soundcalc` reports for every circuit of ZisK v0.16.1 a soundness level of 128 bits under the *Johnson-bound regime* (JBR: list-decoding up to the Johnson radius, a proven regime for Reed–

Solomon codes) and between 111 and 113 bits for the standard AIRs under the more conservative *unique-decoding regime* (UDR), with the recursion circuits at 63–80 bits in UDR owing to their lower rates [Eth26b]. ZisK therefore claims 128-bit security *in the Johnson-bound regime* and does not rely on capacity conjectures for FRI. In the soundness ledger of Section 9 we write $\varepsilon_{\mathrm{be}(N,\rho,q,g)}$ for the knowledge-soundness error of a single backend proof with these parameters; its concrete value is the per-circuit `soundcalc` figure.

Remark (Proof sizes and the compressor layer). *The plain STARK proof of a large AIR is several megabytes (up to* 21 *MiB for Keccakf). Verifying such a proof directly inside a leaf verifier would make the leaf verifier circuit itself large. ZisK therefore inserts, for the six largest AIR families, an intermediate* compressor *circuit that verifies the STARK proof at rate* $1/2$ *and re-proves it at rate* $1/4$ *with far fewer queries; the leaf verifier then verifies the compressor proof instead. The compressor is a recursive verifier whose relation is "the AIR proof verifies and has the given public statement", so it is transparent to the relations of Section 6; we include it in the topology of Section 9.1.*

8.3. Trace Commitments and the Function com(π).

A proof π produced by the backend for an AIR of circuit type τ is a sequence of Merkle roots (one per commitment stage), the out-of-domain evaluations, the FRI folding roots and final polynomial, the Merkle authentication paths for the queried rows, and the grinding nonce. The *trace commitment* of π is

$$\mathrm{com}(\pi) := (\mathrm{root}_1(\pi), \tau),$$

the stage-1 Merkle root together with the identifier of the AIR. Including τ makes commitments of different AIR types distinct messages of the multiset hashed in Section 6.2, so that two segment instances of different types cannot contribute the same element to the multiset; the leaf verifier for type τ has τ hard-coded, so this is not a prover choice. The AIR values $\boldsymbol{a}$ and the public statement are not part of $\mathrm{com}(\pi)$: they are inputs to the verifier and enter the per-proof transcript directly.

Two properties of the trace commitment are used by the soundness sketch. The first is *binding*: the Merkle root of the stage-1 tree binds the prover to the evaluations of the committed columns on L, and by the low-degree test to a unique low-degree witness $\boldsymbol{T}$ (except with the backend soundness error). The second is that the commitment is produced *before* the global challenge α is derived, so that the multiset S of trace commitments hashed in Definition 2 is fixed before α becomes available to any prover. Both properties are stated formally in Section 9.

8.4. **Verification-Key and Cached-Column Ledger.**

The recursion of Section 6 mentions several verification keys with different roles; the review of the W1 draft rightly asked for these to be separated. Table 8 lists them.

Key	**Fixed at**	**Role and how it is bound**
$\mathrm{vk}_{\mathrm{AIR}}^{(\tau)}$, $\tau \in [M]$	setup (per program for AIRs with cached columns)	Verification key of the backend STARK for AIR $\mathcal{A}_\tau$: constraint description, fixed-column root, and stark parameters. Hard-coded into the leaf verifier circuit of type τ (or into the compressor of type τ where one exists). Selected by τ.
$\mathrm{vk}_{\mathrm{LEAF}}^{(\tau)}$, $\tau \in [M]$	setup	Verification key of the leaf verifier circuit of type τ. Hard-coded into the aggregation verifier circuit, which selects it by reading τ from the child's recursive statement.
vk (propagated)	setup, but not hard-coded in the aggregator	The aggregation verifier's own verification key. It cannot be hard-coded into itself, so it is carried as a public through every node; each aggregator uses the value it received to verify aggregation children and passes the same value up.
vk^*	setup	The actual aggregation verifier key, hard-coded into the Final Verifier, which binds the propagated vk to it. This is the fixed parameter of $\mathcal{R}_{\mathrm{FINAL}[\mathrm{vk}^*]}$.
$\mathrm{vk}_{\mathrm{FINAL}}$	setup	Verification key of the Final Verifier; held by the external verifier, together with vk^*, as the *verification context* of ZisK for the given program.
$\mathrm{root}_{\mathrm{ROM}}$	per program	Commitment to the program: the Merkle root of the cached columns of the ROM AIR, which hold the instruction table P (Section 2.2.1). It is bound into $\mathrm{vk}_{\mathrm{AIR}}^{(\mathrm{ROM})}$ and thereby into the whole key chain above, so $\mathrm{vk}_{\mathrm{FINAL}}$ identifies the program.
$\mathrm{root}_{\mathrm{cache}}^{(\tau)}$	setup	Roots of the fixed columns of table AIRs (arithmetic, binary, range, MemAlign, Keccak and SHA-256 tables). They are program-independent, computed deterministically from the table definition, and part of $\mathrm{vk}_{\mathrm{AIR}}^{(\tau)}$.

TABLE 8. Verification-key ledger. "Setup" means computed once for the ZisK release; "per program" means recomputed when the guest program changes. Every key is either hard-coded in a circuit above it or bound by the Final Verifier, so the external verifier's context $(\mathrm{vk}_{\mathrm{FINAL}}, \mathrm{vk}^*)$ transitively fixes every key in the tree.

The distinction between τ and vk is thus the following. The circuit type τ is *dispatch information*: it tells an aggregator which hard-coded child verifier to run, $V_{\mathrm{LEAF}}^{(\tau)}$ for $\tau \in [M]$ or V_{AGG} for $\tau = M + 1$. The propagated vk is *one specific key*, the aggregation verifier's own, which is the only key in the system that cannot be hard-coded where it is needed. In particular the propagated vk is never the key selected by τ; when $\tau \in [M]$ the selected key is $\mathrm{vk}_{\mathrm{LEAF}}^{(\tau)}$, which lives in the aggregator's constants.

For the cached and fixed columns the lifecycle is as follows. Fixed columns are defined by the AIR (they are part of its description) and their evaluations are committed at setup; their correctness is a property of the AIR definition, not of any proof. Cached columns are computed by the prover from the program binary once and committed; their root is bound into the program-specific verification key, and the setup procedure recomputes them deterministically from the binary so that the verifier can check the root against the program it intends to verify. Soundness of lookups against a cached table (e.g. the frequent-operation caches of Section 4.2.5) therefore rests on two facts: that the root in the verification context is the root of the honestly computed table (a setup-time check), and that the STARK binds the prover to that root (backend binding). Both are entries of the proof-obligation ledger in Section 9.5.

9. Soundness: Definitions, Deviations, and Proof Obligations

This section collects the material that a W3 soundness proof of Theorem 3 will be built from: the topology of every proof-system call, a vanilla baseline against which ZisK's design choices are recorded as deviations, the explicit assumption about the transpiler, the lemma chain from an accepting final proof down to a witness of $\mathcal{R}_{\mathrm{exec}}$, and the ledger of proof obligations that remain. Following the EF W2 guidance [Eth26a], nothing here is a complete proof; every step is stated precisely enough that what remains to be proved is visible.

9.1. **Topology of All Proof-System Calls.**

Table 9 lists every circuit for which a STARK proof is produced, its role in the aggregation tree of Section 6, the relation it proves, how many instances of it an execution produces, and the name under which it is registered in the EF `soundcalc` tool [Eth26b]. All circuits are proved with the backend of Section 8; their parameters are in Table 7.

Node kind	**Circuit**	**Relation**	**Instances**	**`soundcalc` name**
AIR leaf	Main	$\mathcal{R}_{\text{AIR}}^{(\text{Main})}$	$\lceil T/2^{22} \rceil$	`Main`
AIR leaf	Rom	$\mathcal{R}_{\text{AIR}}^{(\text{Rom})}$	1	`Rom`
AIR leaf	Mem, RomData, InputData	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	per region workload	`Mem`, `RomData`, `InputData`
AIR leaf	MemAlign, MemAlignByte, MemAlignReadByte, MemAlignWriteByte	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	workload	same names
AIR leaf	Arith, Binary, BinaryAdd, BinaryExtension	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	workload	same names
AIR leaf	Add256, ArithEq, ArithEq384, Keccakf, Sha256f, Poseidon2, Blake2br	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	workload	same names
AIR leaf	the 12 DMA AIRs	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	workload	`Dma`, `DmaMemCpy`, `DmaInputCpy`, `Dma64Aligned`, `Dma64AlignedInputCpy`, `Dma64AlignedMemSet`, `Dma64AlignedMem`, `Dma64AlignedMemCpy`, `DmaUnaligned`, `DmaPrePost`, `DmaPrePostMemCpy`, `DmaPrePostInputCpy`
AIR leaf (table)	SpecifiedRanges, VirtualTable0, VirtualTable1	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	1 each	`SpecifiedRanges`, `VirtualTable0`, `VirtualTable1`
compressor	one per large AIR type	"$V_{\text{AIR}}^{(\tau)}$ accepts"	one per such leaf	`DmaPrePost-compressor`, `ArithEq-compressor`, `ArithEq384-compressor`, `Keccakf-compressor`, `Sha256f-compressor`, `Blake2br-compressor`
leaf verifier	one per AIR type (35)	$\mathcal{R}_{\text{LEAF}}^{(\tau)}$	one per leaf	not registered separately (see remark)
aggregation verifier	Recursive2	$\mathcal{R}_{\text{AGG}}$	$\approx$ (leaves -1) / 2	`Recursive2`
Final Verifier	Final	$\mathcal{R}_{\text{FINAL}[\text{vk}^\star]}$	1	`Final`
Final, compressed	Final_Compressed	"V_{FINAL} accepts"	0 or 1	`Final_Compressed`

TABLE 9. Topology of all proof-system calls in ZisK v0.16.1 and mapping to `soundcalc` circuit names. The 35 AIR leaves are the circuit $\mathcal{C}$ of Table 3. An execution that fits in a single leaf uses the degenerate tree of Section 6 (the Final verifies the leaf verifier directly). The optional compressed Final is a one-to-one re-proof of the Final at a smaller size; the optional wrap into a pairing-based SNARK for on-chain verification is outside the scope of this document.

Remark (Leaf verifiers in soundcalc). *The* `soundcalc` *export produced from the ZisK proving key registers the AIRs, the compressors, the aggregation verifier and the two Final circuits, but not the per-type leaf verifier circuits. Their parameters are those of the aggregation verifier family (rate* 1/8*,* 73 *queries,* 20 *grinding bits), so the security level reported for* `Recursive2` *applies to them; making them explicit entries is a pending item of the* `soundcalc` *integration.*

Every auxiliary argument —memory, lookup, permutation, range check, continuation— is realised *inside* the AIR leaves as bus interactions and not as a separate proof-system invocation. Their public interface between proof instances is therefore uniform: every leaf exposes exactly its bus contribution $s \in \mathbb{K}$ and its trace commitment (through $\boldsymbol{c}$), and the global relation that ties the instances together is $\mathcal{R}^{\alpha}_{\mathrm{BUS}}$ of Definition 6, checked as $s = 0$ at the Final. The semantic relation each bus enforces is given in Section 4.4 and Section 5.3.

9.2. Vanilla Baseline and Deviation Ledger.

We adopt the baseline-and-deviation strategy of the W2 requirements [Eth26a]. The *vanilla construction* we compare against is the following: a single monolithic AIR whose trace is the whole execution, proved by a plain DEEP-ALI STARK with FRI; memory checked by a sorted-permutation argument inside that AIR with Merkle-committed memory at segment boundaries; all lookups and permutations realised by LogUp with independent verifier challenges α, γ sampled per proof; fixed segmentation of the execution into whole-VM-state checkpoints proved separately and stitched by equality of boundary states; and binary aggregation of proofs by a recursive verifier that hard-codes every verification key. This is essentially the reference vanilla zkVM of the EF sprint and its security argument (correct-trace extractability by descending the recursion tree) is the one we intend to adapt. Table 10 lists every proof-system-level difference between the vanilla construction and ZisK, with the layer it affects and the argument by which soundness is preserved.

Deviation	**Layer**	**Why soundness is preserved (sketch)**	**Interacts with**
D1. Per-chip trace splitting instead of whole-VM segmentation (Section 5.2.1)	segmentation	No whole-state equality is available; instead the Main's boundary state is a typed payload on a continuation bus and the memory is checked *globally* by one sorted trace spanning all segments. Soundness reduces to Lemma 3 for the Main and to Lemma 3 for memory.	D2, D4
D2. LogUp bus balancing across independently proved AIRs, checked only at the root (Definition 6)	PIOP / recursion	Each leaf's partial sum is a public output bound by its own STARK; the root check $s = 0$ is the LogUp identity for the union of all interactions. Soundness is the LogUp bound at a random α; the challenge must be common to all leaves, which is D3.	D3, D6
D3. Global challenge from an LtHash multiset hash of all trace commitments, supplied to leaves as a public input and justified at the root (Definition 2)	transcript / recursion	Commit-then-justify: leaves use α before it is justified, but every leaf's commitment enters the multiset whose hash the Final recomputes. Changing any commitment changes $\mathcal{H}(S)$ except with ε_{ms}, hence α, hence invalidates the leaf. Requires Assumption 3 and the random-oracle modelling of DeriveAlpha.	D2, D8
D4. Compression challenge γ derived from α instead of sampled independently	PIOP	$\gamma = H_\gamma(\alpha)$ with H_γ a random oracle is a fresh random value given α; the LogUp collision bound applies with (α, γ) modelled as a random pair determined by the transcript.	D2
D5. Circuit-type dispatch τ and propagated vk (Section 8.4)	recursion	τ is hard-coded by the circuit that produced the proof and selects a hard-coded key in the parent; vk propagates unchanged and is fixed at the root. An accepting aggregation with a wrong vk at any node would require an accepting Final with a child declaring $\text{vk} \neq \text{vk}^*$, which the Final rejects.	D3
D6. Registers handled by a linked-step argument in Main instead of the sorted memory (Section 4.2.1.1)	AIR / bus	Timestamp-linked chain with exact multiset matching on the Memory Bus is a standard alternative memory-checking argument; per-register chain is opened once globally and reloaded at every segment boundary. Register refinement obligation in Section 9.5.	D1
D7. MemAlign decomposition of subword accesses into aligned ones (Section 4.2.2.4)	AIR / bus	Fixed micro-program table; write-back stamped in the following slot preserves order. Lemma 4.	D6
D8. Precompiles access memory directly, ordered by the dispatching step (Section 4.4.1)	AIR / bus	The Main places its step t on the Operation Bus; the precompile stamps its accesses with slots $2, 3$ of t, so the memory argument orders them exactly where the ISA semantics places the precompile's effect.	D1, D6
~~D11.~~ [illegible]	~~AIR~~ / [illegible]	[illegible]	~~D6~~, D7
[illegible]	PIOP / recursion	[illegible]	[illegible]
[illegible] (Section 4.4.4)	[illegible] ment	[illegible] those of P with $\text{Commit}_{\text{ROM}(P)} = \text{root}_{\text{ROM}}$.	

TABLE 10. Deviation ledger: proof-system-level differences between the vanilla construction and ZisK v0.16.1.

9.3. **Transpiler Correctness Assumption.**

ZisK executes programs in its own ISA, obtained from a RISC-V binary P_{RV} by a deterministic transpiler Transpile (Section 2). Everything proved in this document concerns the ZisK program $P = \mathrm{Transpile}(P_{\mathrm{RV}})$, which is what $\mathrm{root}_{\mathrm{ROM}}$ commits to. A claim about the RISC-V binary requires the following separate assumption, which is part of ZisK's trusted computing base and not discharged by any proof.

Assumption 1 (Transpiler Correctness). For every RISC-V binary P_{RV} in the supported subset, every input in and every hint sequence h: if the ZisK execution of $\mathrm{Transpile}(P_{\mathrm{RV}})$ on (in, h) is successful and declares outputs out, then the RISC-V execution of P_{RV} on in with the same hint responses terminates successfully and produces the same public outputs out.

Under Assumption 1, Theorem 3 yields the statement about the original program: an accepting proof for $x = \left(\mathrm{Commit}_{\mathrm{ROM}(\mathrm{Transpile}(P_{\mathrm{RV}}))}, \mathrm{out}\right)$ implies that P_{RV} has a successful execution producing out on some input and hints. The verifier must independently know that $\mathrm{root}_{\mathrm{ROM}}$ is the commitment of the transpiled binary it cares about; in deployments this is established once, at setup, by recomputing $\mathrm{Commit}_{\mathrm{ROM}}$ from the binary.

9.4. **Soundness Sketch: From an Accepting Final Proof to an Execution.**

We sketch the extractor $\mathcal{E}_{\mathrm{fin}}$ of Theorem 3 and the lemma each step needs. Throughout, "accepting" means the relevant verifier outputs accept, and every step is conditioned on the failure events of the previous steps not occurring; the bound of Theorem 3 is the union bound over all failure events.

Step 1 (Final layer). Run the straight-line knowledge extractor of the Final's STARK on $(\boldsymbol{p}, \pi)$ to obtain a witness $(\alpha, \sigma, \pi_{\mathrm{AGG}})$ of $\mathcal{R}_{\mathrm{FINAL}[\mathrm{vk}^*]}$: an aggregation proof accepted by V_{AGG} under $(\boldsymbol{p}, \alpha, \sigma, \mathrm{vk}^*)$ with $s = 0$, $\kappa < B$, $\alpha = \mathrm{DeriveAlpha}(\boldsymbol{p}, \boldsymbol{c})$. *Fails with* $\varepsilon_{\mathrm{be(Final)}}$.

Step 2 (Recursive descent). Apply the extractors of V_{AGG} and $V_{\mathrm{LEAF}}^{(\tau)}$ recursively: an accepting aggregation proof yields k accepting child proofs with statements σ_i summing to σ and all carrying vk^*; an accepting leaf-verifier proof of type τ yields an accepting AIR proof π_τ with s_τ and $\boldsymbol{c}_\tau = h(\mathrm{com}(\pi_\tau))$. Because every child's vk equals the parent's and the root's is vk^*, every aggregation proof in the tree was verified with the genuine aggregation key (deviation D5); because κ is range-checked at every node and every node's κ is the sum of its children's, the tree has $\kappa < B$ leaves. *Fails with* $\sum_v \varepsilon_{\mathrm{be}(v)}$ over internal nodes. This step is the analogue of the "combine tree unrolling" lemma of the vanilla analysis; well-foundedness follows from κ strictly increasing towards the root.

Lemma 2 (Recursive descent). From an accepting Final proof, the extractor obtains a multiset of at most B accepting AIR proofs $\{(\tau_\ell, \boldsymbol{p}, \alpha, s_\ell, \pi_\ell)\}_\ell$ such that $\sum_\ell s_\ell = 0$ and $\alpha = \mathrm{DeriveAlpha}\big(\boldsymbol{p}, \sum_\ell h(\mathrm{com}(\pi_\ell))\big)$, except with probability $\sum_v \varepsilon_{\mathrm{be}(v)}$.

Step 3 (AIR extraction). For each leaf, run the STARK extractor to obtain a witness $(\boldsymbol{T}_\ell, \boldsymbol{a}_\ell)$ satisfying $\mathcal{A}_{\tau_\ell}$, consistent with $\boldsymbol{p}$, with $\mathrm{BusCont}_\alpha\big(\mathcal{I}_{\tau_\ell}, \boldsymbol{T}_\ell, \boldsymbol{a}_\ell\big) = s_\ell$, and whose stage-1 commitment is $\mathrm{com}(\pi_\ell)$. *Fails with* $\sum_\ell \varepsilon_{\mathrm{be}(\tau_\ell)} + \varepsilon_{\mathrm{bind}}$.

Step 4 (Challenge soundness). The multiset $S = \{\mathrm{com}(\pi_\ell)\}$ of extracted commitments was fixed before α was available to any prover: each $\mathrm{com}(\pi_\ell)$ is the first message of its proof, and α is a function of $\boldsymbol{p}$ and $\mathcal{H}(S)$. By Assumption 3, no other multiset of size at most B has the same hash except with $\varepsilon_{\mathrm{ms}}$; modelling DeriveAlpha as a random oracle, α is therefore uniformly random and independent of the extracted witnesses. This is the formal content of the commit-then-justify argument for α. *Fails with* $\varepsilon_{\mathrm{ms}} + \varepsilon_{\mathrm{FS}}$.

Step 5 (Bus soundness). The extracted witnesses form a witness for the circuit $\mathcal{C}$ (possibly with several instances of each AIR) whose bus contributions sum to zero at a uniformly random α (Step 4). By the LogUp soundness bound of Section 3, every bus is balanced in the sense of Definition 3 except with $\varepsilon_{\mathrm{LogUp}} \le \frac{N_{\mathrm{tot}} + \ell N_{\mathrm{tot}}^2}{|\mathbb{K}|}$. Hence the per-bus semantic relations hold: Definition 6 (every Main row fetches $P[\mathrm{pc}]$), Definition 5 (every delegated operation is correct), the table-bus membership relations, and the continuation relations.

Step 6 (Stitching and memory). By Lemma 3 applied to the Main's continuation bus, the Main instances form one sequence $G_1, ..., G_m$ starting at $(\mathrm{pc}_{\mathrm{BOOT}}, 0)$ and ending at $(\mathrm{pc}_{\mathrm{END}}, 0)$, with matching (pc, c) at every boundary; concatenating their traces gives a sequence of rows $r = 0, ..., T-1$ with globally consistent step counter, program counter and c. By Lemma 3 and its refinements, the multiset of memory operations requested by all rows (Main, MemAlign, precompiles) is consistent with the initial memory mem_0; define mem_t as the memory obtained by applying, in memory-step order, all operations with memory step less than $1 + 4t$. This is the deterministic *memory reconstruction rule* of the extractor. The read-only data region's contents are the first-access values, and the input-data region's contents define in.

Step 7 (AIR-to-step). Set $S_t := (\mathrm{pc}_t, c_t, \mathrm{mem}_t)$ from the Main rows and the reconstructed memory, $\iota_t := P[\mathrm{pc}_t]$ (Step 5, ROM relation) and $h_t :=$ the values read from the free-input port at step t. The Main's constraints, together with the operation-bus relation (correct (c, flag) and correct precompile memory effects) and the memory relation (operand

reads return mem_t's values and the store updates mem_{t+1}), imply $\mathrm{Step}_P(S_t, \iota_t, h_t, S_{t+1})$ for every t. This is a property of the Main AIR's constraint system alone and is the central W3 obligation.

Lemma 3 (AIR-to-step). Let a Main row r satisfy the Main AIR, let its ROM Bus message be a row of P, let its Operation Bus message (if any) be valid in the sense of Definition 5, and let its Memory Bus messages be consistent with mem_t in the sense of Step 6. Then the state read off row $r+1$ equals Step_P applied to the state read off row r with the row's hints.

Step 8 (Boundary and outputs). The genesis and terminus anchors give $S_0 = (\mathrm{pc}_{\mathrm{BOOT}}, 0, \mathrm{mem}_0)$ and $\mathrm{pc}_T = \mathrm{pc}_{\mathrm{END}}$, $c_T = 0$. The global `pubout` terms on the Operation Bus, balanced in Step 5, give exactly one `pubout` per index i in the Main trace with value out_i from $\boldsymbol{p}$, establishing $\mathrm{Bind}_{\mathrm{out}}$. The ROM relation with $\mathrm{root}_{\mathrm{ROM}}$ from $\boldsymbol{p}$ establishes ProgramBind. Hence $(x; w_{\mathrm{exec}}) \in \mathcal{R}_{\mathrm{exec}}$ with w_{exec} the extracted execution.

The union of the failure probabilities of Steps 1–7 is the bound of Theorem 3. Steps 1–3 are instances of backend knowledge soundness; Step 4 is the LtHash and Fiat-Shamir assumptions; Steps 5–8 are properties of ZisK's constraint system and buses and constitute the ZisK-specific W3 proof obligations.

9.5. **Proof-Obligation Ledger.**

Table 11 lists every lemma or assumption consumed by the sketch above, with its status.

Id	Obligation	Status
O1	Knowledge soundness of the backend STARK for each circuit of Table 9 with the parameters of Table 7, in the random-oracle model for Poseidon2; straight-line extraction and its composition across the recursion (idealised, Section 7.3).	standard / assumed; parameters audited by `soundcalc`
O2	Binding of the stage-1 trace commitment and of the fixed and cached column commitments (Merkle trees over Poseidon2).	standard, from collision resistance
O3	LtHash $(B, \varepsilon_{\text{ms}})$-binding with $n = 372$, $B = 2^{20}$ (Assumption 3); consistency of the convention $\kappa < B$ everywhere.	assumed (SIS); convention fixed in this revision
O4	Fiat-Shamir: DeriveAlpha$(\boldsymbol{p}, \mathcal{H}(S))$ modelled as a random oracle on (statement, multiset); the per-proof STARK challenges likewise.	assumed (random oracle)
O5	LogUp soundness: if the union of all extracted witnesses does not balance a bus in the sense of Definition 3, then $\sum_i s_i = 0$ holds with probability at most $\frac{N_{\text{tot}}+\ell N_{\text{tot}}^2}{\lvert\mathbb{K}\rvert}$ over (α, γ); with $\gamma = H_\gamma(\alpha)$ (D4).	standard, adapted to D4; W3
O6	Well-formedness of every bus message: the interaction polynomials of each AIR emit exactly the schemas of Section 4.4, with binary selectors, and no AIR emits a message on a bus it is not listed on.	inspection of the PIL sources; W3
O7	Memory argument (Lemma 3): the constraints of Definition 2 imply consistency of the requested operations with the initial memory; including the bounded-increment argument and the bridging rows.	new (sorted-permutation memory over LogUp); W3
O8	Register refinement: the linked-step argument of Section 4.2.1.1 with the per-segment reload implies that every register read returns the last written value, registers start at zero, and the 2^{24} window is never exceeded.	new; W3
O9	MemAlign refinement (Lemma 4) and byte-level MemAlign variants.	new; W3
O10	Binding of the read-only data region's initial contents and of any RAM initialisation to the program committed by root_{ROM} (remark in Section 4.2.2.1): identify the chip that requests the initialising writes at v0.16.1 and show it is bound to root_{ROM}.	open at v0.16.1; resolved by the ROM AIR's init messages in later versions
O11	Stitching (Lemma 3) for the Main and for each Memory region; the `last`-flag argument that excludes cycles.	new; W3
O12	AIR-to-step (Lemma 3): the Main AIR's constraints implement operand resolution, store, and control flow exactly as in Section 2.3, including the `set_pc`/`flag` exclusion and the 32-bit address formation.	new; W3 (constraint audit)
O13	Per-executor correctness: each executor and precompile AIR accepts an Operation Bus message only if it is valid per Appendix B and its memory effect; frequent-operation	W3 (per-chip constraint audit)
O14	[illegible]	[illegible]
O15	[illegible]	[illegible]
O16	[illegible]	[illegible]
O17	[illegible]	[illegible]

TABLE 11. Proof-obligation ledger. "Standard" obligations are instances of published results for the named primitive; "new" ones are specific to ZisK's design and constitute the W3 work; "open" marks the single item whose status at v0.16.1 this revision could not settle from the specification alone.

Acknowledgments

This paper draws on the designs presented in the OpenVM [Ope25] and Nexus [Nex24] whitepapers, which served as important sources of inspiration. We also acknowledge RISC Zero [RIS23] as a pioneering zkVM and SP1 [Suc24] for advancing the practical deployment of RISC-V-based proving systems.

We extend our gratitude to the Polygon zkEVM and Plonky3 [Pol24] teams for their foundational work in zero-knowledge proving systems, as well as to the RISC-V community [RIS19] for providing a robust architecture that enables the zkVM model.

References

GWC19. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge, https://eprint.iacr.org/2019/953

Ben+18. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In: Chatzigiannakis, I., Kaklamanis, C., Marx, D., and Sannella, D. (eds.) ICALP 2018: 45th International Colloquium on Automata, Languages and Programming. p. 14:1–14:17. Schloss Dagstuhl (2018a)

DP23. Diamond, B.E., Posen, J.: Succinct Arguments over Towers of Binary Fields, https://eprint.iacr.org/2023/1784

HLP24. Haböck, U., Levit, D., Papini, S.: Circle STARKs, https://eprint.iacr.org/2024/278

Bel+23. Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J.L., Rubio, A., Baylina, J.: Circom: A Circuit Description Language for Building Zero-Knowledge Applications, (2023)

Ele22. Electric Coin Company: Halo2, https://github.com/zcash/halo2, (2022)

GPR21. Goldberg, L., Papini, S., Riabzev, M.: Cairo – a Turing-complete STARK-friendly CPU architecture, https://eprint.iacr.org/2021/1063

Lit24. Lita Foundation: Valida: a STARK-based virtual machine focusing on code reuse, https://github.com/valida-xyz/valida, (2024)

RIS19. RISC-V International: The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. (2019)

RIS23. RISC Zero: RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity, (2023)

Suc24. Succinct Labs: SP1: A performant, open-source zkVM, (2024)

Nex24. Nexus: The Nexus zkVM, (2024)

Ben+18. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable, transparent, and post-quantum secure computational integrity, https://eprint.iacr.org/2018/046

RIS24. RISC-V International: RISC-V ELF psABI Specification, (2024)

Hab22. Haböck, U.: Multivariate lookups based on logarithmic derivatives, https://eprint.iacr.org/2022/1530

FS87. Fiat, A., Shamir, A.: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology – CRYPTO'86. pp. 186–194. Springer, Berlin, Heidelberg (1987)

Ope25. OpenVM Contributors: OpenVM Whitepaper, https://openvm.dev/whitepaper.pdf, (2025)

Blu+91. Blum, M., Evans, W.S., Gemmell, P., Kannan, S., Naor, M.: Checking the Correctness of Memories. In: 32nd Annual Symposium on Foundations of Computer Science. pp. 90–99. IEEE Computer Society Press (1991)

Hem+25. Hemo, T., Jue, K., Rabinovich, E., Roh, G., Rothblum, R.D.: Jagged Polynomial Commitments (or: How to Stack Multilinears), https://eprint.iacr.org/2025/917

Cla+03. Clarke, D.E., Devadas, S., Dijk, M. van, Gassend, B., Suh, G.E.: Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In: Laih, C.-S. (ed.) Advances in Cryptology – ASIACRYPT~2003. pp. 188–207. Springer, Berlin, Heidelberg (2003)

Zis25. ZisK: Secure Challenge Derivation in ZisK, https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg

GKS23. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A Faster Version of the Poseidon Hash Function. In: El Mrabet, N., De Feo, L., and Duquesne, S. (eds.) AFRICACRYPT 23: 14th International Conference on Cryptology in Africa. pp. 177–203. Springer, Cham (2023)

Ajt96. Ajtai, M.: Generating Hard Instances of Lattice Problems (Extended Abstract). In: 28th Annual ACM Symposium on Theory of Computing. pp. 99–108. ACM Press (1996)

Eth26. Ethereum Foundation Cryptography Team: Towards a zkVM architecture whitepaper; W2 soundness requirements for zkVM submissions, (2026a)

BCG24. Barbara, A., Chiesa, A., Guan, Z.: Relativized Succinct Arguments in the ROM Do Not Exist, https://eprint.iacr.org/2024/728

Eth26. Ethereum Foundation Cryptography Research: soundcalc: security evaluation of inner-segment arguments of zkVMs, https://github.com/ethereum/soundcalc

Sta23. StarkWare: ethSTARK Documentation – Version 1.2, https://eprint.iacr.org/2021/582

BCS16. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive Oracle Proofs. In: Theory of Cryptography – TCC 2016-B, Part II. pp. 31–60. Springer, Berlin, Heidelberg (2016)

Pol24. Polygon: Plonky3, (2024)

Appendix

A. Arguments

This appendix gives the formal statements of the argument types used in ZisK's arithmetization framework. All are realised through the same bus-balancing primitive of Section 3.4 —one or more sender interactions with positive multiplicity, one or more receiver interactions with negative multiplicity, on a shared bus— but they differ in the semantic relationship they enforce on the messages.

Throughout this appendix we fix a circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{M}$ with witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}_{i=1}^{M}$, and we write $m(r), \boldsymbol{e}(r) \in \mathbb{F}^{\ell}$ for the row-r evaluation of an interaction $(b, m, \boldsymbol{e})$ under its witness.

Permutation Argument.

A *permutation argument* enforces that two AIRs observe the same multiset of messages on a shared bus. It is the natural primitive whenever data is reordered —e.g., between unsorted and sorted views of the same memory accesses— but not modified.

Definition (Permutation Argument). Let $\iota_1 = (b, m_1, \boldsymbol{e}_1) \in \mathcal{I}_{i_1}$ and $\iota_2 = (b, -m_2, \boldsymbol{e}_2) \in \mathcal{I}_{i_2}$ be a sender/receiver pair of interactions of common length ℓ on a shared bus b, with boolean multiplicities $m_1, m_2 \in \{0, 1\}$. The pair (ι_1, ι_2) realises a *permutation argument* in the sense that the joint contribution of ι_1 and ι_2 to bus b vanishes if and only if

$$\left\{\boldsymbol{e}_1(r) : r \in \left[N_{i_1}\right], m_1(r) = 1\right\} = \left\{\boldsymbol{e}_2(r) : r \in \left[N_{i_2}\right], m_2(r) = 1\right\}$$

as multisets.

Lookup Argument.

A *lookup argument* enforces that every message sent on the bus by a "query" AIR also appears as a row of a "table" AIR, with the table free to receive each query any number of times.

Definition (Lookup Argument). Let $\iota_q = \left(b, m_q, \boldsymbol{e}_q\right) \in \mathcal{I}_{i_q}$ be a *query* interaction with boolean multiplicity $m_q \in \{0, 1\}$, and $\iota_t = (b, -m_t, \boldsymbol{e}_t) \in \mathcal{I}_{i_t}$ a *table* interaction with $m_t(r) \geq 0$ counting how many queries land on row r of the table. Both interactions have common length ℓ. The pair $\left(\iota_q, \iota_t\right)$ realises a *lookup argument* in the sense that their joint contribution to bus b vanishes if and only if

$$\forall r \in \left[N_{i_q}\right] \text{ with } m_q(r) = 1 : \exists r' \in \left[N_{i_t}\right] \text{ s.t. } \boldsymbol{e}_q(r) = \boldsymbol{e}_t(r').$$

Lookup arguments underpin most of ZisK's table-driven constraints: instruction decoding, byte and limb decompositions, and any check whose

efficient form is "the output of a sub-chip must belong to a precomputed table".

Range Check Argument.

A *range check argument* enforces that an expression takes values in a bounded interval. It is a specialisation of the lookup argument against a fixed interval table, but it is exposed as a first-class primitive because its receiving side is a shared system table rather than an AIR-specific one.

Definition (Range Check Argument). Fix integers $x \leq y$ and let b be the bus designated for the range check on $[x, y]$. Let $\iota = (b, m, e) \in \mathcal{I}_i$ be an interaction of length $\ell = 1$ with boolean multiplicity $m \in \{0, 1\}$, paired with a receiver interaction $\iota_t = (b, -m_t, e_t)$ on a dedicated table AIR whose trace materialises the set $\{x, x + 1, ..., y\}$ and whose multiplicity $m_t(r) \geq 0$ counts how many incoming queries land on each value. The pair (ι, ι_t) realises a *range check argument* on $[x, y]$ in the sense that their joint contribution to bus b vanishes if and only if

$$\forall r \in [N_i] \quad \text{with} \quad m(r) = 1 : e(r) \in [x, y].$$

In practice ZisK shares each range-check table across the entire circuit: a single $[x, y]$ table absorbs queries from every AIR that needs that range, so the marginal cost of a range check is one row of querying plus one row of multiplicity bookkeeping in the shared table.

Copy Argument.

A *copy argument* asserts that a designated set of cells in a single AIR's trace hold equal values. It is the wiring primitive used inside an AIR to relate values that must coincide but live in different rows or columns —for instance, feeding the output of one row back as the input of a later row, or forcing two columns to agree on a shared intermediate value.

Definition (Copy Argument). Let $\mathcal{A}$ be an AIR with witness $(\boldsymbol{T}, \boldsymbol{a})$, and let $C = \{(r_1, j_1), ..., (r_q, j_q)\}$ be a finite set of *cell coordinates* in $\boldsymbol{T}$, each (r, j) identifying the cell at row r of column j. Let $\{C_1, ..., C_t\}$ be a partition of C into *equivalence classes*. A *copy argument* for $(C, \{C_\ell\})$ enforces:

$$\forall \ell \in [t], \forall (r, j), (r', j') \in C_\ell : T_{j,r} = T_{j',r'}.$$

Operationally, a copy argument is realised as a permutation argument applied class-by-class: for each C_ℓ, the values at all cells in C_ℓ are sent on a dedicated bus and required to coincide as multisets with the same values shifted by an arbitrary cycle on C_ℓ —this forces every pair of cells in the class to agree.

B. Instruction Set

This appendix lists every opcode of the ZisK ISA at v0.16.1, grouped by the chip that proves it (Section 4.2). It is a transcription of `zisk_ops.rs` at that version and is the normative definition of the operation $(c, \text{flag}) = \text{op}(a, b)$ used by the step predicate of Section 2.3.

We write $a_{32} = a \bmod 2^{32}$ for the low word of a, $\text{sext}_k(x)$ for the sign extension of a k-bit value to 64 bits, $\underset{u}{<}, \underset{s}{<}$ (resp. $\underset{u}{\leq}, \underset{s}{\leq}$) for unsigned and signed comparison, and $[\varphi] \in \{0, 1\}$ for the indicator of φ. All arithmetic on 64-bit words is modulo 2^{64} and on 32-bit words modulo 2^{32} unless stated otherwise. Each entry reads "mnemonic (code): c; flag", with flag $= 0$ when omitted. Every `_w` opcode depends only on the low 32 bits of its operands.

Internal Opcodes (Main).

`flag` (`0x00`) $c = 0$; flag $= 1$. Used for unconditional jumps.

`copyb` (`0x01`) $c = b$.

`halt` (`0xff`) the invalid-instruction sentinel; it sets an error and cannot appear in a valid execution trace.

Binary Opcodes (Binary and BinaryAdd).

64-bit:

`minu` (`0x02`), `min` (`0x03`) $c = \min_u(a, b)$, resp. $\min_s(a, b)$.

`maxu` (`0x04`), `max` (`0x05`) $c = \max_u(a, b)$, resp. $\max_s(a, b)$.

`ltu` (`0x06`), `lt` (`0x07`) $c = \left[a \underset{u}{<} b\right]$, resp. $\left[a \underset{s}{<} b\right]$; flag $= c$.

`eq` (`0x09`) $c = [a = b]$; flag $= c$.

`add` (`0x0a`) $c = a + b$ (BinaryAdd chip).

`sub` (`0x0b`) $c = a - b$.

`leu` (`0x0c`), `le` (`0x0d`) $c = \left[a \underset{u}{\leq} b\right]$, resp. $\left[a \underset{s}{\leq} b\right]$; flag $= c$.

`and` (`0x0e`), `or` (`0x0f`), `xor` (`0x10`) bitwise $a \wedge b$, $a \vee b$, $a \oplus b$.

32-bit (results sign-extended from bit 31):

`minu_w` (`0x12`), `min_w` (`0x13`) $c = \text{sext}_{32}(\min_u(a_{32}, b_{32}))$, resp. with $\min_s$.

`maxu_w` (`0x14`), `max_w` (`0x15`) $c = \text{sext}_{32}(\max_u(a_{32}, b_{32}))$, resp. with $\max_s$.

`ltu_w` (`0x16`), `lt_w` (`0x17`) $c = \left[a_{32} \underset{u}{<} b_{32}\right]$, resp. $\left[a_{32} \underset{s}{<} b_{32}\right]$; flag $= c$.

`eq_w` (`0x19`) $c = [a_{32} = b_{32}]$; flag $= c$.

`add_w` (`0x1a`), `sub_w` (`0x1b`) $c = \text{sext}_{32}((a_{32} \pm b_{32}) \bmod 2^{32})$.

`leu_w` (`0x1c`), `le_w` (`0x1d`) $c = \left[a_{32} \underset{u}{\leq} b_{32}\right]$, resp. $\left[a_{32} \underset{s}{\leq} b_{32}\right]$; flag $= c$.

Binary Extension Opcodes (BinaryExtension).

`sll` (`0x21`), `srl` (`0x22`), `sra` (`0x23`) $c = a \ll (b \bmod 64)$, logical $a \gg (b \bmod 64)$, arithmetic $a \gg (b \bmod 64)$.

`sll_w` (`0x24`), `srl_w` (`0x25`), `sra_w` (`0x26`) $c = \text{sext}_{32}$ of the corresponding 32-bit shift of a_{32} by $b \bmod 32$.

`signextend_b` (`0x27`), `signextend_h` (`0x28`), `signextend_w` (`0x29`) $c = \text{sext}_8(b)$, $\text{sext}_{16}(b)$, $\text{sext}_{32}(b)$; a is ignored.

Arithmetic Opcodes (Arith).

`mulu` (`0xb0`), `mul` (`0xb4`) $c = a \cdot b \bmod 2^{64}$ (identical for the unsigned and signed readings).

`muluh` (`0xb1`), `mulsuh` (`0xb3`), `mulh` (`0xb5`) $c =$ the high 64 bits of the 128-bit product of a and b read as unsigned×unsigned, signed×unsigned, signed×signed.

`mul_w` (`0xb6`) $c = \mathrm{sext}_{32}(a_{32} \cdot b_{32} \bmod 2^{32})$.

`divu` (`0xb8`), `remu` (`0xb9`) unsigned $c = \lfloor a/b \rfloor$, resp. $a \bmod b$; if $b = 0$ then $c = 2^{64} - 1$, resp. $c = a$, and flag $= 1$.

`div` (`0xba`), `rem` (`0xbb`) signed quotient rounded toward zero, resp. remainder with the sign of a; if $b = 0$ then $c = 2^{64} - 1$, resp. $c = a$, and flag $= 1$; the overflow case $(-2^{63})/(-1)$ yields -2^{63}, resp. 0.

`divu_w` (`0xbc`), `remu_w` (`0xbd`) $c = \mathrm{sext}_{32}$ of the 32-bit unsigned quotient, resp. remainder, of a_{32} and b_{32}; if $b_{32} = 0$ then $c = 2^{64} - 1$, resp. $c = \mathrm{sext}_{32}(a_{32})$, and flag $= 1$.

`div_w` (`0xbe`), `rem_w` (`0xbf`) $c = \mathrm{sext}_{32}$ of the 32-bit signed quotient, resp. remainder, with the same zero-divisor and overflow conventions.

Public Output and Hint Opcodes (Main).

`pubout` (`0x30`) $c = b$; declares public output word $a \in [0, 32)$ to be b (Section 7). Bound globally on the Operation Bus.

`fcall_param` (`0xf6`), `fcall` (`0xf7`), `fcall_get` (`0xf8`) $c = b$; host-side hint bookkeeping only (Section 2.2.8). The hint value itself is obtained by a subsequent load from the free-input port.

Precompile Opcodes.

For every precompile, $a = 0$ (immediate) and b is the address of an 8-byte-aligned parameter block, unless stated otherwise; $c = 0$ and flag $= 0$ unless stated otherwise. The memory effect $\mathrm{Eff}_{\mathrm{op}}$ of Section 2.3 is the update described for each entry; all pointers are read from the parameter block at $\mathrm{mem}[b + 8i]_8$.

`keccak` (`0xf1`, Keccakf chip) $\mathrm{mem}[b, b + 200) \leftarrow$ Keccak-f[1600]$(\mathrm{mem}[b, b + 200))$, the state read and written in place.

`sha256` (`0xf9`, Sha256f chip) parameter block $(\text{state_ptr}, \text{input_ptr})$; the 256-bit state at state_ptr is replaced by one SHA-256 compression of the 512-bit block at input_ptr.

`poseidon2` (`0xe1`, Poseidon2 chip) $\mathrm{mem}[b, b + 128)$, read as 16 field elements, is replaced by its Poseidon2 permutation of width 16.

`blake2` (`0xea`, Blake2br chip) parameter block $(\text{index}, \text{state_ptr}, \text{input_ptr})$ with index a direct value; one BLAKE2b round on the 16-word state at state_ptr with the 16-word message block at input_ptr.

`add256` (`0xf0`, Add256 chip) parameter block $(x_{\text{ptr}}, y_{\text{ptr}}, \text{cin}, z_{\text{ptr}})$ with cin direct; $\text{mem}\big[z_{\text{ptr}}, z_{\text{ptr}} + 32\big) \leftarrow x + y + \text{cin} \bmod 2^{256}$ and $c =$ the carry out $\in \{0, 1\}$.

`arith256` (`0xf2`, ArithEq chip) parameter block (x, y, z, d_l, d_h) of pointers to 256-bit values; $d_h \cdot 2^{256} + d_l \leftarrow x \cdot y + z$.

`arith256_mod` (`0xf3`, ArithEq chip) parameter block (x, y, z, m, d); $d \leftarrow (x \cdot y + z) \bmod m$.

`secp256k1_add` (`0xf4`), `secp256r1_add` (`0xe8`), `bn254_curve_add` (`0xfa`) (ArithEq chip) parameter block (p_1, p_2) of pointers to affine points; $p_1 \leftarrow p_1 + p_2$ on the named curve.

`secp256k1_dbl` (`0xf5`), `secp256r1_dbl` (`0xe9`), `bn254_curve_dbl` (`0xfb`) (ArithEq chip) b points directly to an affine point p; $p \leftarrow 2p$.

`bn254_complex_add` (`0xfc`), `bn254_complex_sub` (`0xfd`), `bn254_complex_mul` (`0xfe`) (ArithEq chip) parameter block (f_1, f_2) of pointers to $\mathbb{F}_{p^2}$ elements over the BN254 base field; $f_1 \leftarrow f_1 \circ f_2$.

`arith384_mod` (`0xe2`, ArithEq384 chip) as `arith256_mod` over 384-bit values.

`bls12_381_curve_add` (`0xe3`), `bls12_381_curve_dbl` (`0xe4`) (ArithEq384 chip) as the 256-bit curve operations, on BLS12-381 with 384-bit coordinates.

`bls12_381_complex_add` (`0xe5`), `bls12_381_complex_sub` (`0xe6`), `bls12_381_complex_mul` (`0xe7`) (ArithEq384 chip) as the BN254 $\mathbb{F}_{p^2}$ operations, over the BLS12-381 base field.

DMA Opcodes (Dma chips).

Unlike the other precompiles, the DMA opcodes take a and b as plain scalars. The byte count n comes from the word at the fixed system address `EXTRA_PARAMS` for the plain variants and from the extended argument (the `j1` field) for the `x` variants.

`dma_memcpy` (`0xd0`), `dma_xmemcpy` (`0xd6`) $\text{mem}[a, a + n) \leftarrow \text{mem}[b, b + n)$; $c = a$.

`dma_memcmp` (`0xd1`), `dma_xmemcmp` (`0xd7`) $c =$ the three-way comparison of $\text{mem}[a, a + n)$ and $\text{mem}[b, b + n)$; memory unchanged.

`dma_xmemset` (`0xd9`) $\text{mem}[a, a + b) \leftarrow$ the fill byte given by the extended argument; $c = a$.

`dma_inputcpy` (`0xd2`) copies b bytes from the input-data region to a; $c = a$.

`_dma_64_aligned` (`0xda`), `_dma_unaligned` (`0xdb`), `_dma_pre` (`0xdc`), `_dma_post` (`0xdd`) internal opcodes into which the DMA front-end decomposes a bulk operation; they never appear in a program and exist only as Operation Bus messages between the DMA AIRs.